

Algoritmi e Strutture Dati

Capitolo 8

Il problema della Coda con priorità

Tipo di dato CodaPriorità (1/2)

tipo CodaPriorita:

dati:

un insieme S di n elementi di tipo *elem* a cui sono associate chiavi di tipo *chiave* prese da un universo totalmente ordinato.

operazioni:

`findMin()` $\rightarrow$ *elem*

restituisce l'elemento in S con la chiave minima.

`insert(elem e, chiave k)`

aggiunge a S un nuovo elemento e con chiave k .

`delete(elem e)` 

cancella da S l'elemento e .

Suppongo sempre che mi venga dato un riferimento diretto all'elemento da cancellare

`deleteMin()`

cancella da S l'elemento con chiave minima.

Tipo di dato CodaPriorità (2/2)

Operazioni aggiuntive

`increaseKey(elem  $e$ , chiave  $d$ )`

incrementa della quantità d la chiave dell'elemento e in S .

`decreaseKey(elem  $e$ , chiave  $d$ )`

decrementa della quantità d la chiave dell'elemento e in S .

`merge(CodaPriorita  $c_1$ , CodaPriorita  $c_2$ )  $\rightarrow$  CodaPriorita`
restituisce una nuova coda con priorità $c_3 = c_1 \cup c_2$.

Applicazioni: gestione code in risorse condivise, gestione priorità in processi concorrenti, progettazione di algoritmi efficienti per diversi problemi fondamentali (es: calcolo cammini minimi in un grafo, minimo albero ricoprente, ordinamento, ecc.)

Quattro implementazioni elementari

1. Array non ordinato
2. Array ordinato
3. Lista non ordinata
4. Lista ordinata

Ci focalizzeremo soltanto sulle **operazioni di base**.

Array non ordinato

Lo dimensiono sufficientemente grande e tengo traccia del numero **n** di elementi nella coda in una variabile di appoggio

- **FindMin**: $\Theta(n)$ (devo guardare tutti gli elementi)
- **Insert**: $O(1)$ (inserisco in coda)
- **Delete**: $O(1)$ (poiché mi viene fornito il riferimento diretto all'elemento da cancellare, lo posso cancellare in $O(1)$ sovracopiando l'ultimo elemento)
- **DeleteMin**: $\Theta(n)$ (devo prima cercare il minimo in $\Theta(n)$, poi lo posso cancellare in $O(1)$)

Array ordinato

Lo dimensiono sufficientemente grande, lo tengo **ordinato** in ordine **decrescente** e tengo traccia del numero **n** di elementi nella coda in una variabile di appoggio

- **FindMin**: $O(1)$
- **Insert**: $O(n)$ (trovo in $\Theta(\log n)$ la giusta posizione, ma poi devo fare $O(n)$ spostamenti)
- **Delete**: $O(n)$ (devo fare $O(n)$ spostamenti)
- **DeleteMin**: $O(1)$ (l'elemento minimo è in fondo all'array, non devo fare spostamenti)

Lista non ordinata

La considero **bidirezionale**



- **FindMin**: $\Theta(n)$ (devo guardare tutti gli elementi)
- **Insert**: $O(1)$ (inserisco in coda o in testa)
- **Delete**: $O(1)$ (poiché mi viene fornito il riferimento diretto all'elemento da cancellare, lo posso cancellare in $O(1)$ agendo sui puntatori)
- **DeleteMin**: $\Theta(n)$ (devo prima cercare il minimo in $\Theta(n)$, poi lo posso cancellare in $O(1)$)

Lista ordinata

Tengo la lista bidirezionale **ordinata** in ordine **crescente**

- **FindMin**: $O(1)$ (il minimo è in testa alla lista)
- **Insert**: $O(n)$ (trovo in $O(n)$ la giusta posizione, e poi faccio in $O(1)$ l'inserimento)
- **Delete**: $O(1)$ (agisco sui puntatori)
- **DeleteMin**: $O(1)$ (basta far puntare la testa della lista al secondo elemento della lista stessa)

Riepilogo implementazioni elementari

	FindMin	Insert	Delete	DeleteMin
Array non ord.	$\Theta(n)$	$O(1)$	$O(1)$	$\Theta(n)$
Array ordinato	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Lista non ordinata	$\Theta(n)$	$O(1)$	$O(1)$	$\Theta(n)$
Lista ordinata	$O(1)$	$O(n)$	$O(1)$	$O(1)$

Tre implementazioni evolute

- ➔ d-heap: generalizzazione degli heap binari visti per l'ordinamento
- ➔ Heap binomiali
- ➔ Heap di Fibonacci (cenni)



d-heap

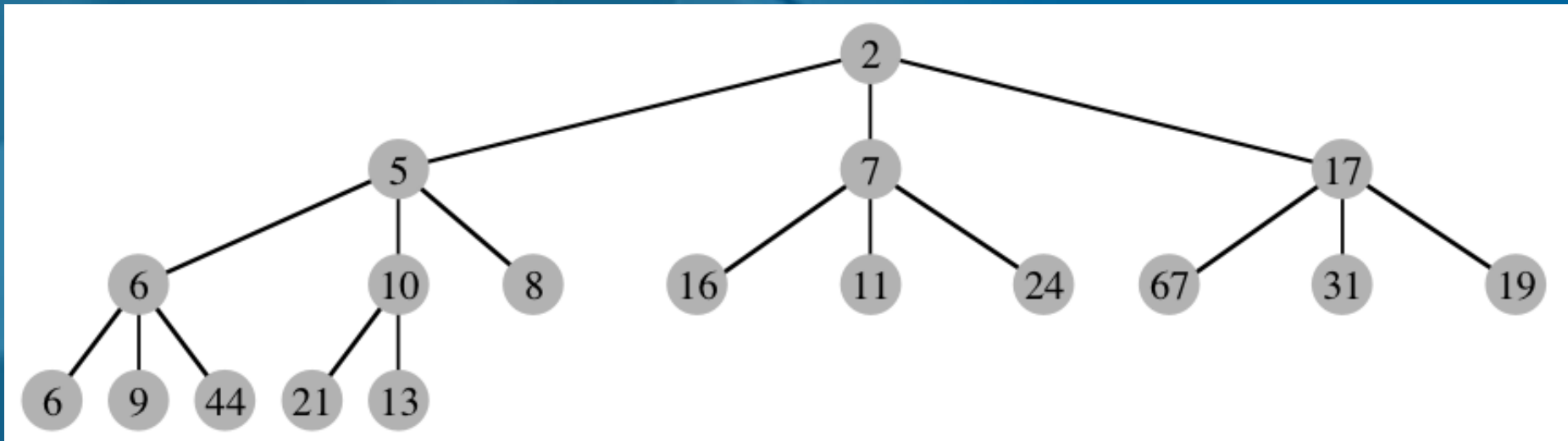
Definizione

Un d-heap è un albero radicato d-ario con le seguenti proprietà:

1. **Struttura**: è completo almeno fino al penultimo livello, e tutte le foglie sull'ultimo livello sono compattate verso sinistra
2. **Contenuto informativo**: ogni nodo v contiene un elemento $\text{elem}(v)$ ed una chiave $\text{chiave}(v)$ presa da un dominio totalmente ordinato
3. **Ordinamento parziale (inverso) dell'heap (min-heap)**: $\text{chiave}(v) \geq \text{chiave}(\text{parent}(v))$ per ogni nodo v diverso dalla radice

Esempio

Heap d-ario con 18 nodi e $d=3$



Proprietà

1. Un d-heap con n nodi ha **altezza** $\Theta(\log_d n)$
2. La **radice** contiene l'**elemento con chiave minima** (per via della proprietà di ordinamento a heap)
3. Può essere **rappresentato implicitamente** tramite vettore posizionale grazie alla proprietà di struttura

Procedure ausiliarie

Utili per ripristinare la proprietà di ordinamento a heap su un nodo v che non la soddisfi

$$T(n) = O(\log_d n)$$

procedura *muoviAlto*(v)

while ($v \neq \text{radice}(T)$ **and** $\text{chiave}(v) < \text{chiave}(\text{padre}(v))$) **do**
 scambia di posto v e $\text{padre}(v)$ in T

il vecchio amico *FixHeap*!

procedura *muoviBasso*(v)

$$T(n) = O(d \log_d n)$$

repeat

 sia u il figlio di v con la minima $\text{chiave}(u)$, se esiste

if (v non ha figli o $\text{chiave}(v) \leq \text{chiave}(u)$) **break**

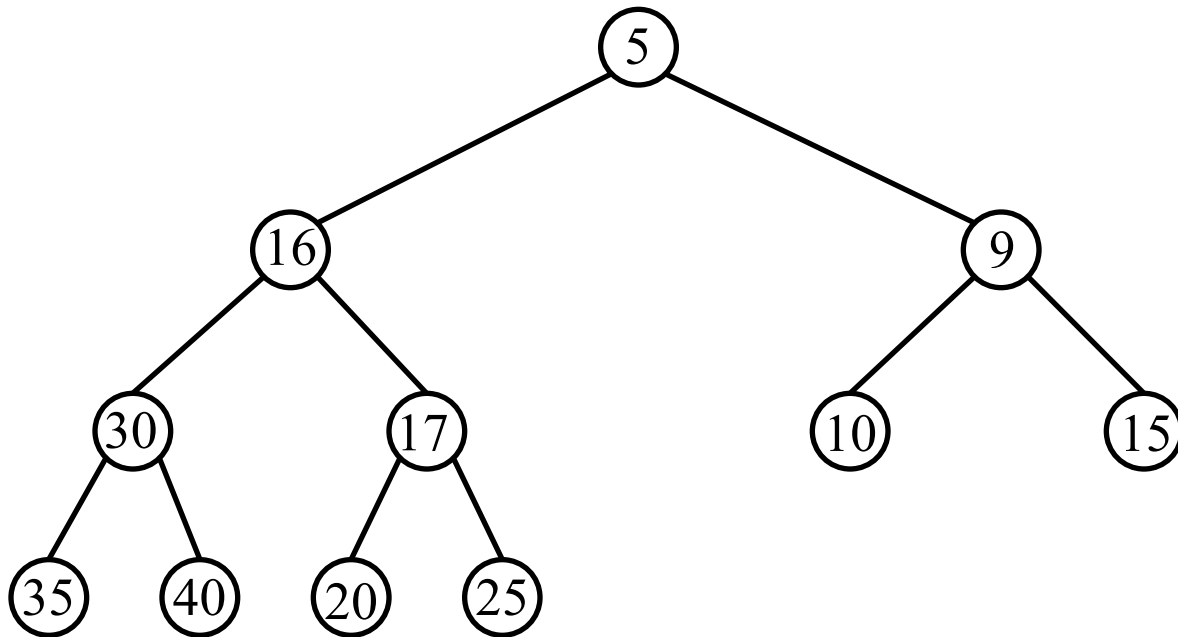
 scambia di posto v e u in T

findMin

`findMin()` $\rightarrow$ *elem*
restituisce l'elemento nella radice di T .

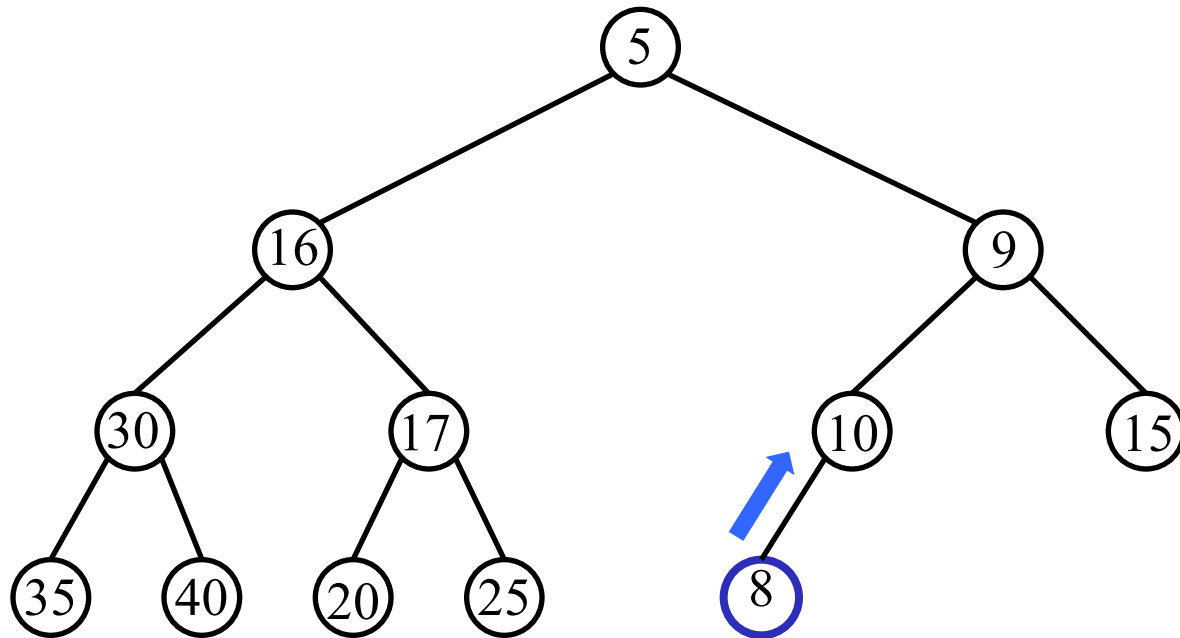
$$T(n)=O(1)$$

`insert(elem e, chiave k)`



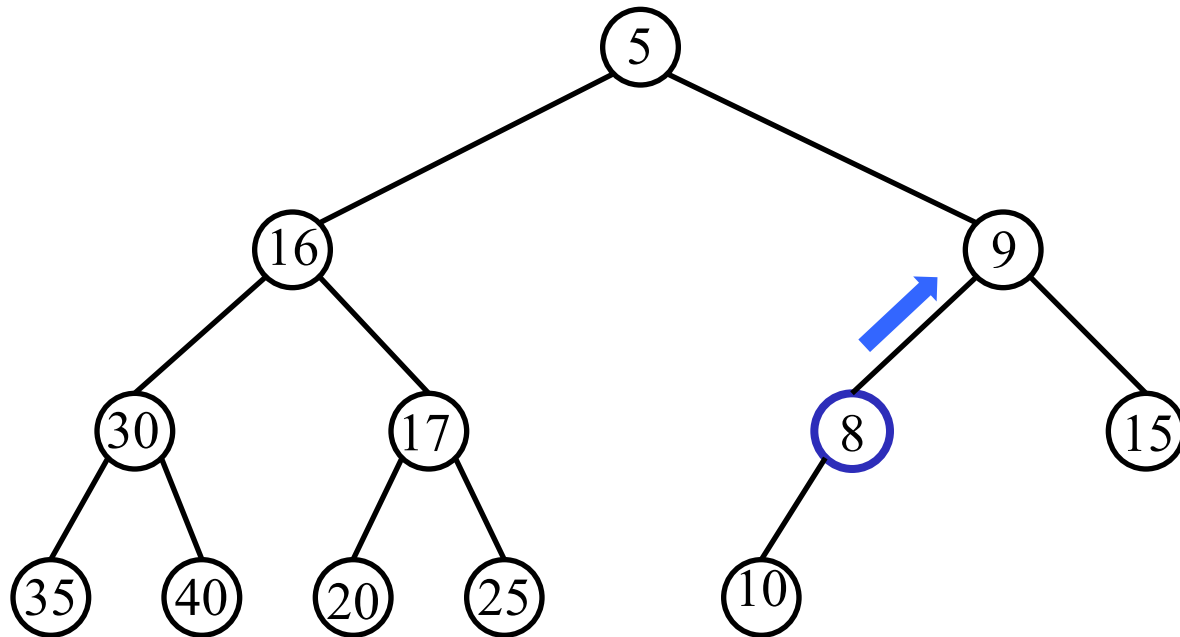
`Insert(e,8)`

insert(elem e, chiave k)



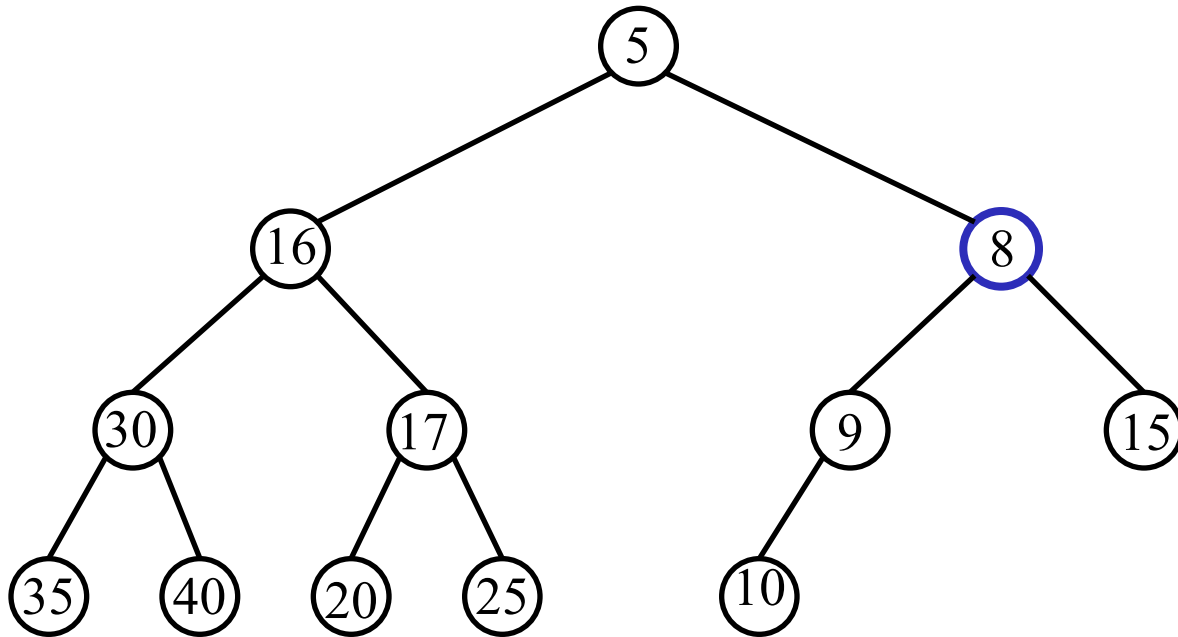
Insert(e,8)

insert(elem e, chiave k)



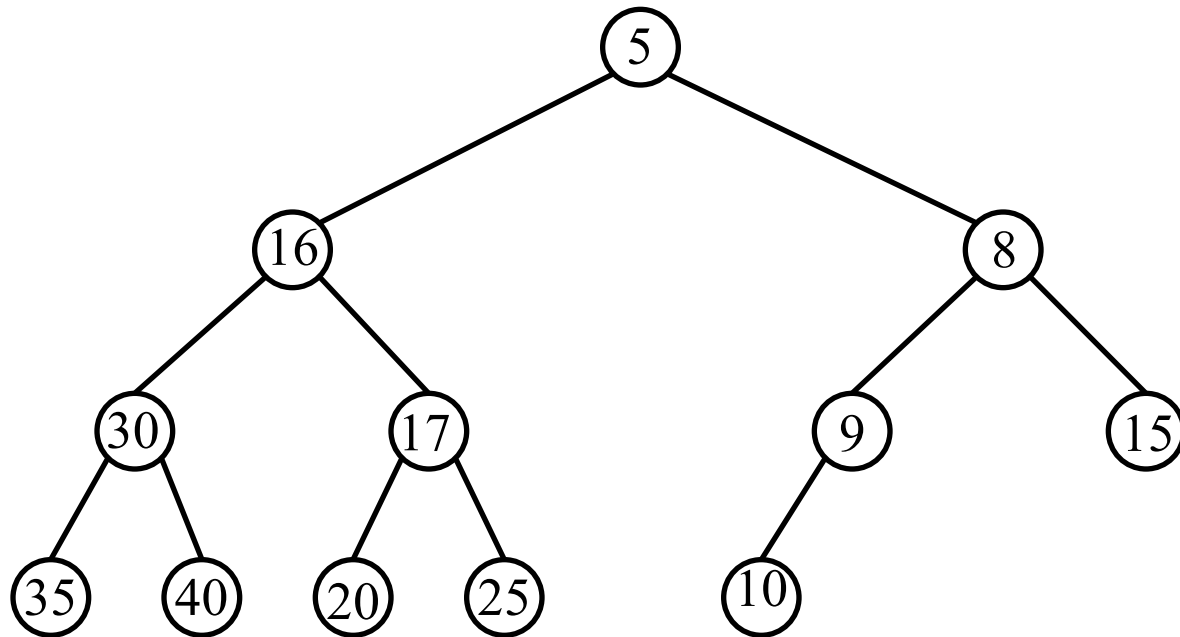
Insert(e,8)

insert(elem e, chiave k)



Insert(e,8)

insert(elem e, chiave k)



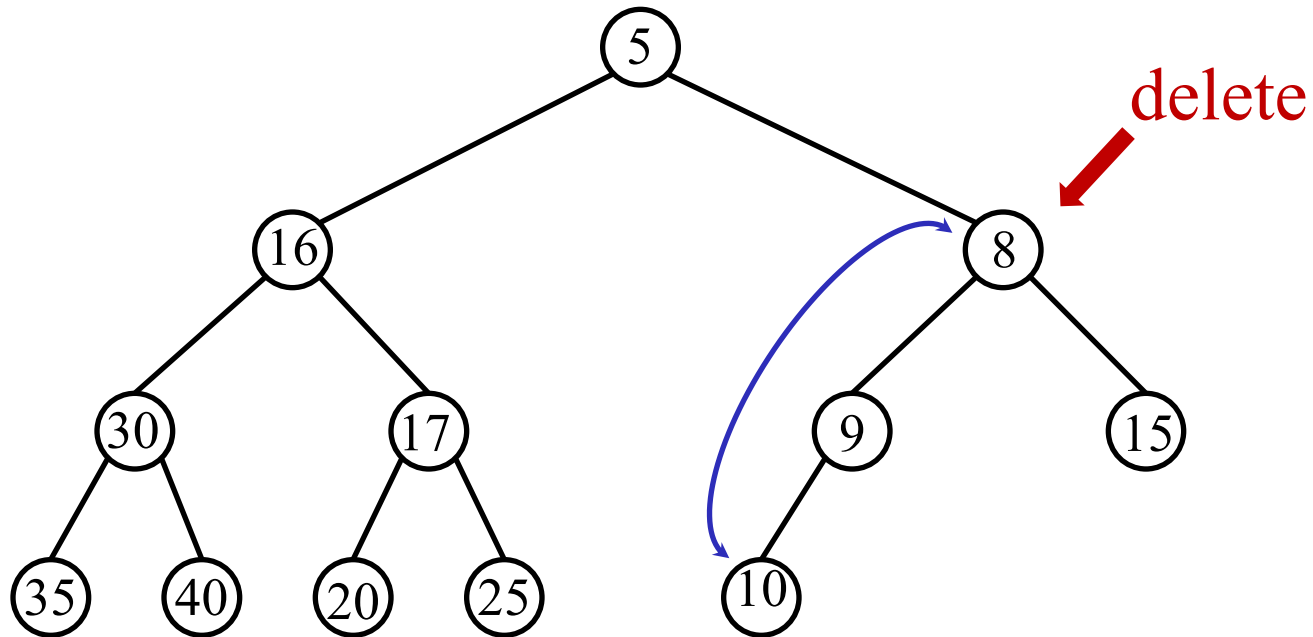
Insert(e,8)

insert(elem e , chiave k)

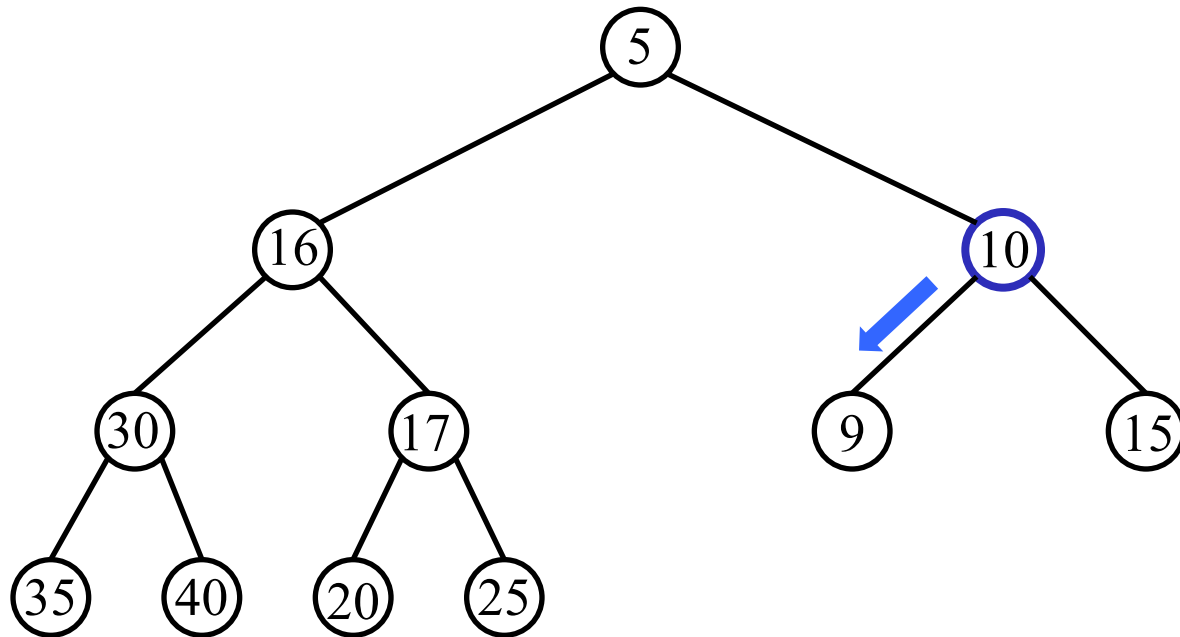
crea un nuovo nodo v con elemento e e chiave k , in modo che diventi una foglia sull'ultimo livello di T . La proprietà dell'ordinamento a heap viene poi ripristinata spingendo il nodo v verso l'alto tramite ripetuti scambi di nodi.

$T(n)=O(\log_d n)$ per l'esecuzione di muoviAlto

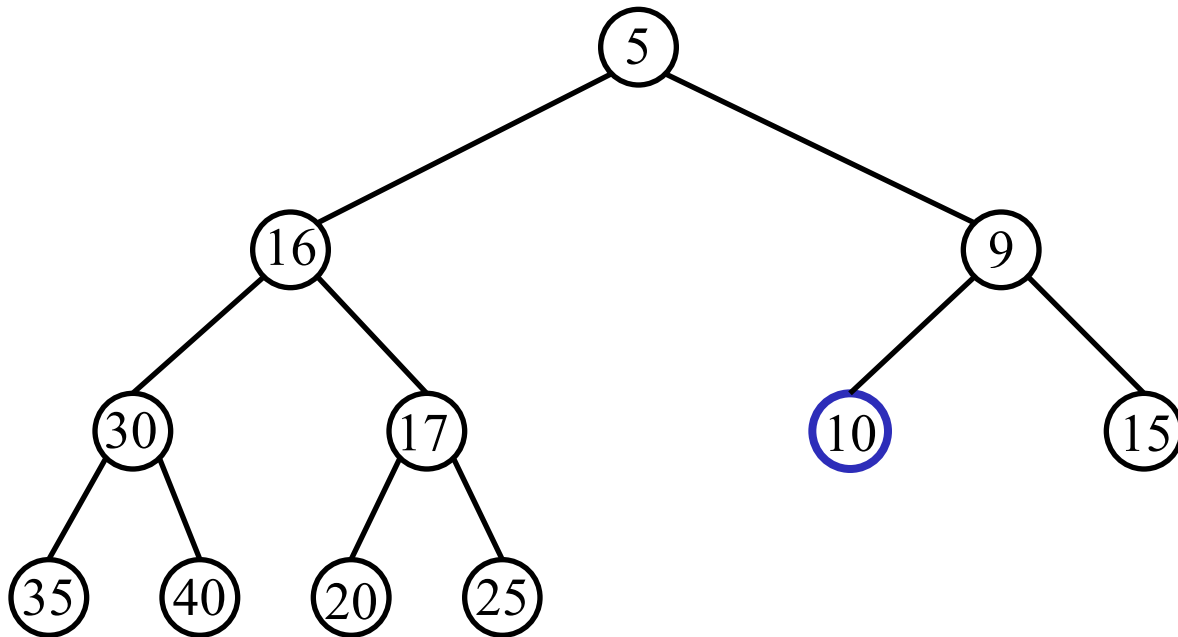
delete(elem e) e deleteMin



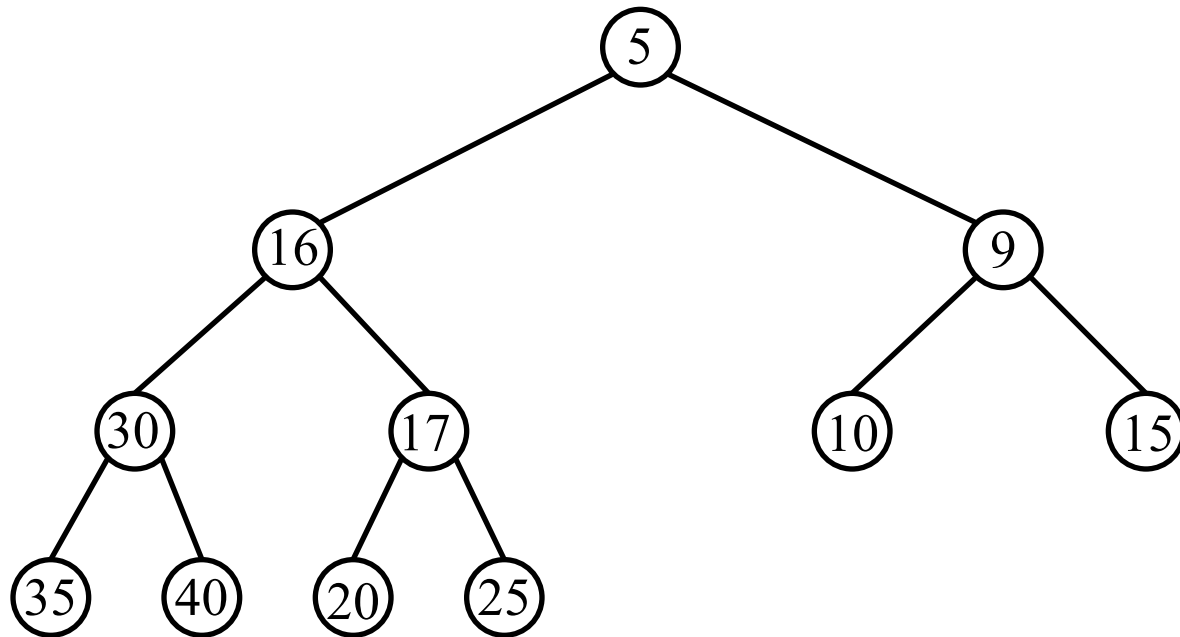
delete(elem e) e deleteMin



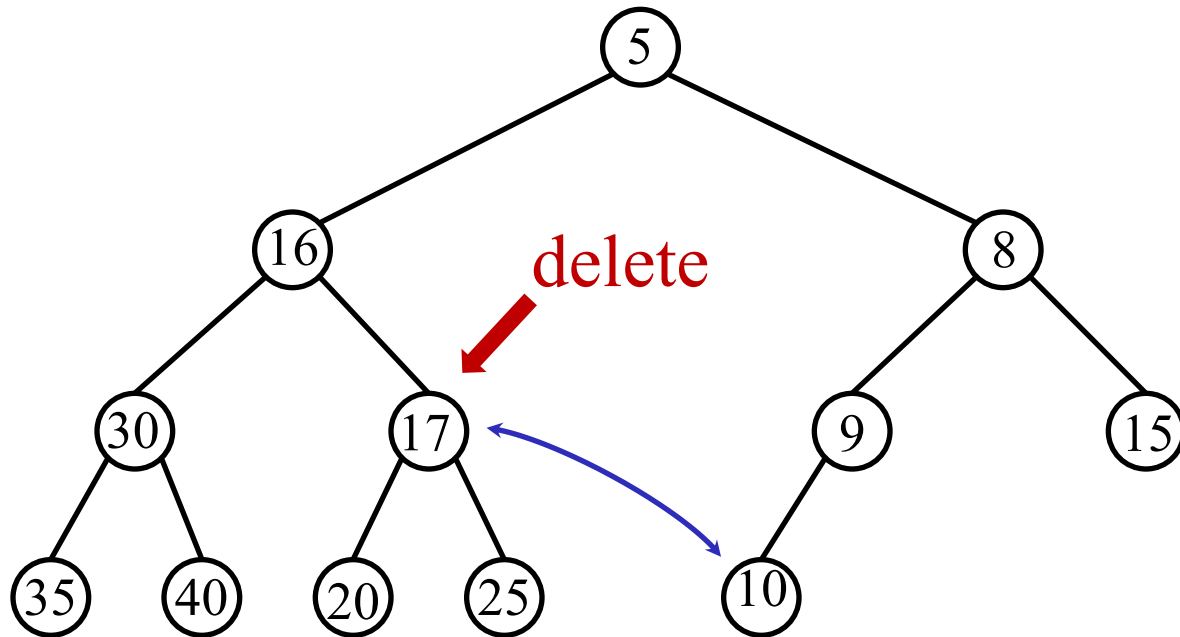
delete(elem e) e deleteMin



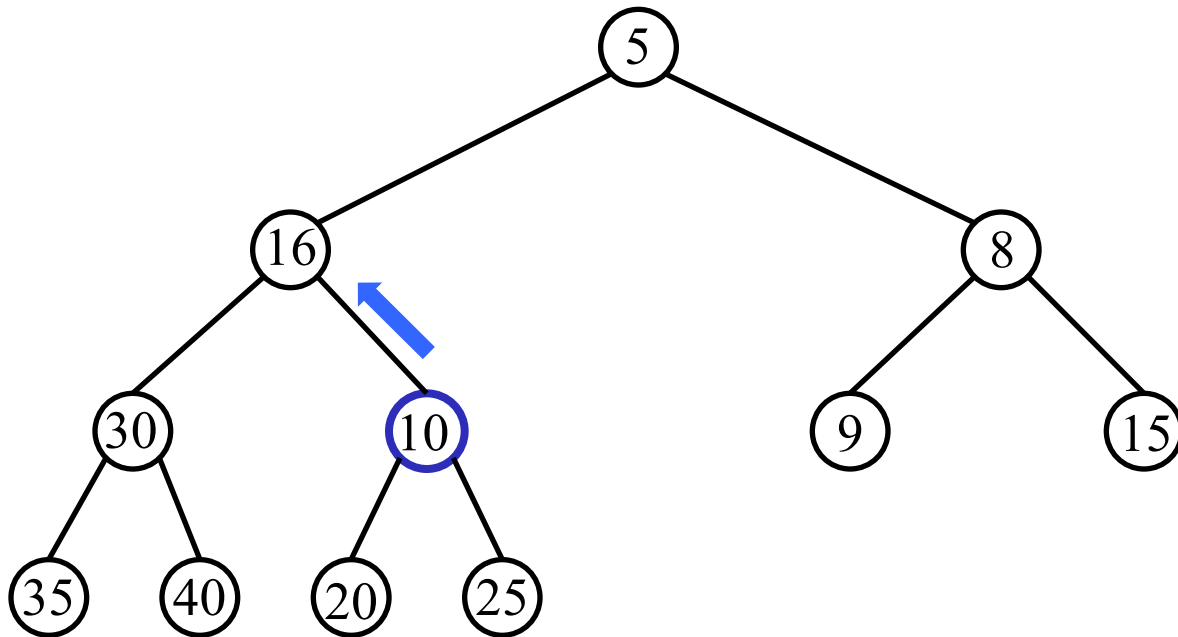
delete(elem e) e deleteMin



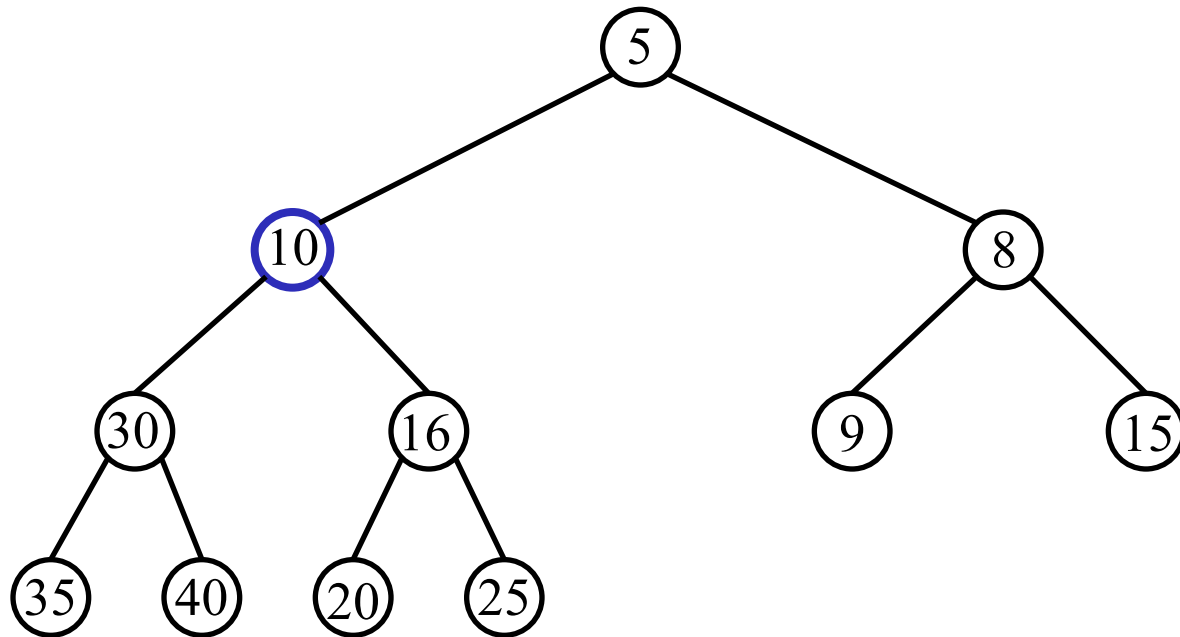
delete(elem e) e deleteMin



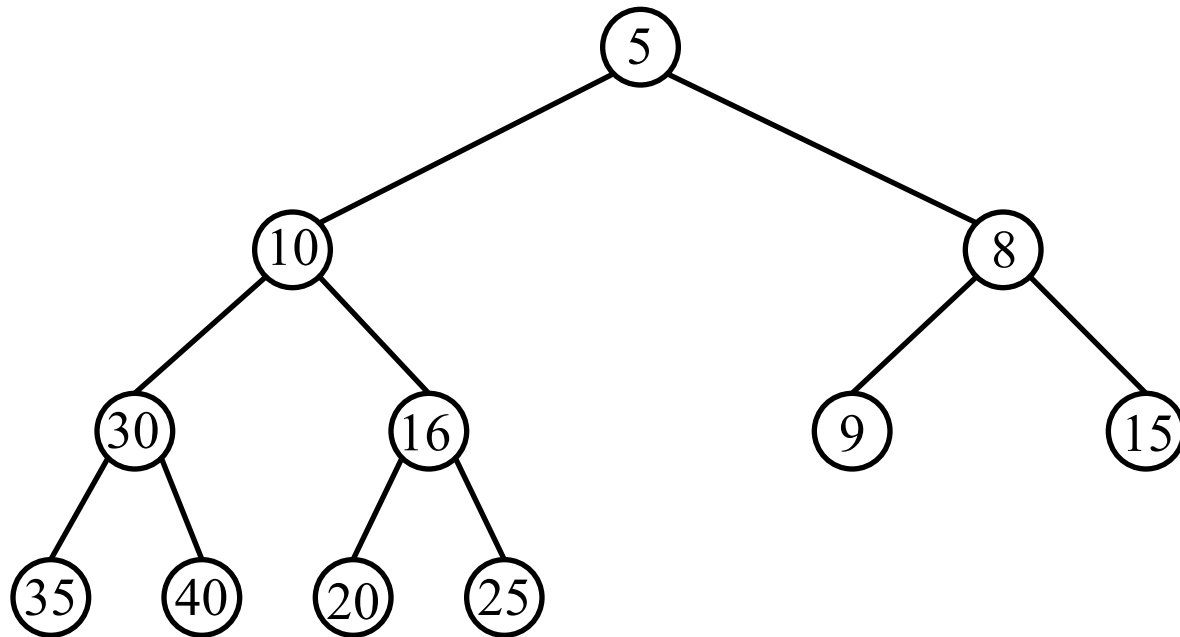
delete(elem e) e deleteMin



delete(elem e) e deleteMin



delete(elem e) e deleteMin



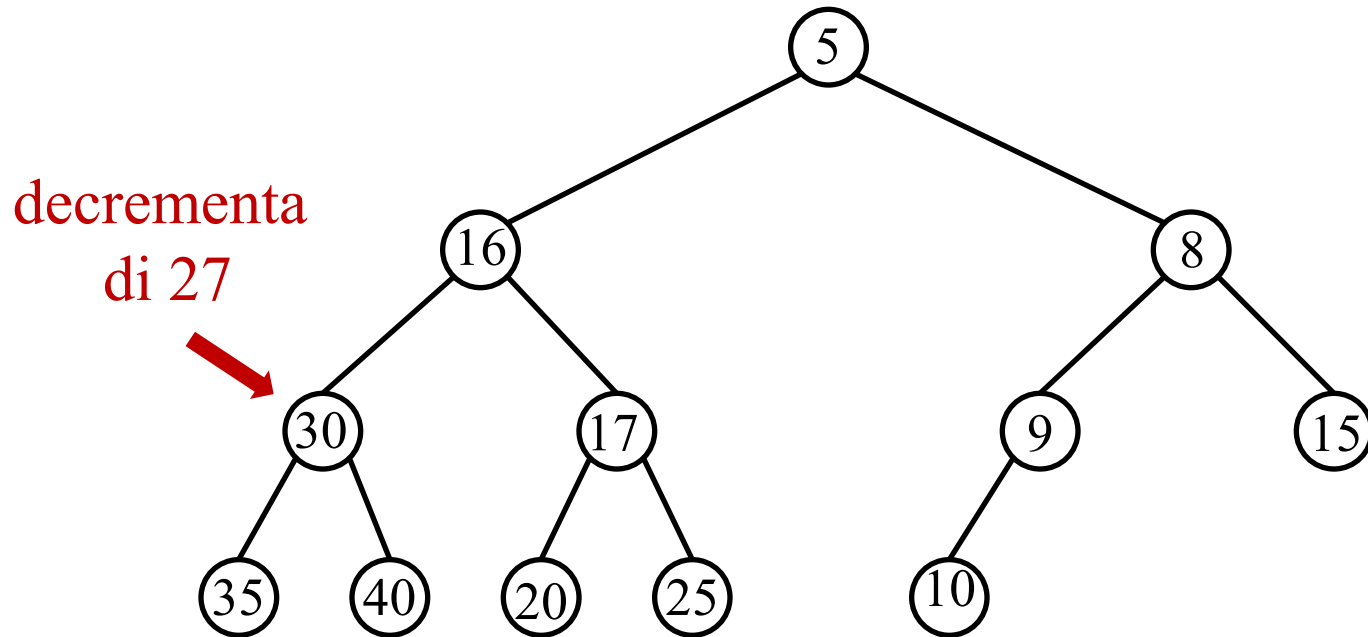
delete(elem e) e deleteMin

scambia il nodo v contenente l'elemento e con una qualunque foglia u sull'ultimo livello di T , e poi elimina v . Ripristina infine la proprietà dell'ordinamento a heap spingendo il nodo u verso la sua posizione corretta scambiandolo ripetutamente con il proprio padre o con il proprio figlio contenente la chiave più piccola

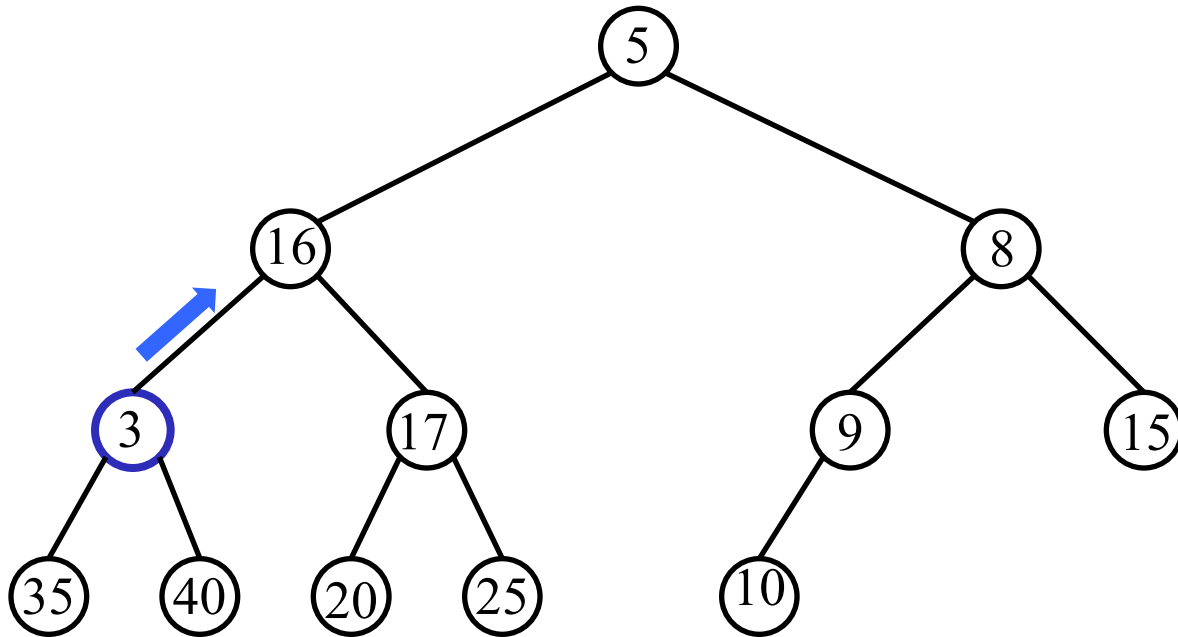
$T(n) = O(\log_d n)$ o $O(d \log_d n)$ per l'esecuzione di `muoviAlto` o `muoviBasso`

Può essere usata anche per implementare la cancellazione del minimo, con costo $O(d \log_d n)$

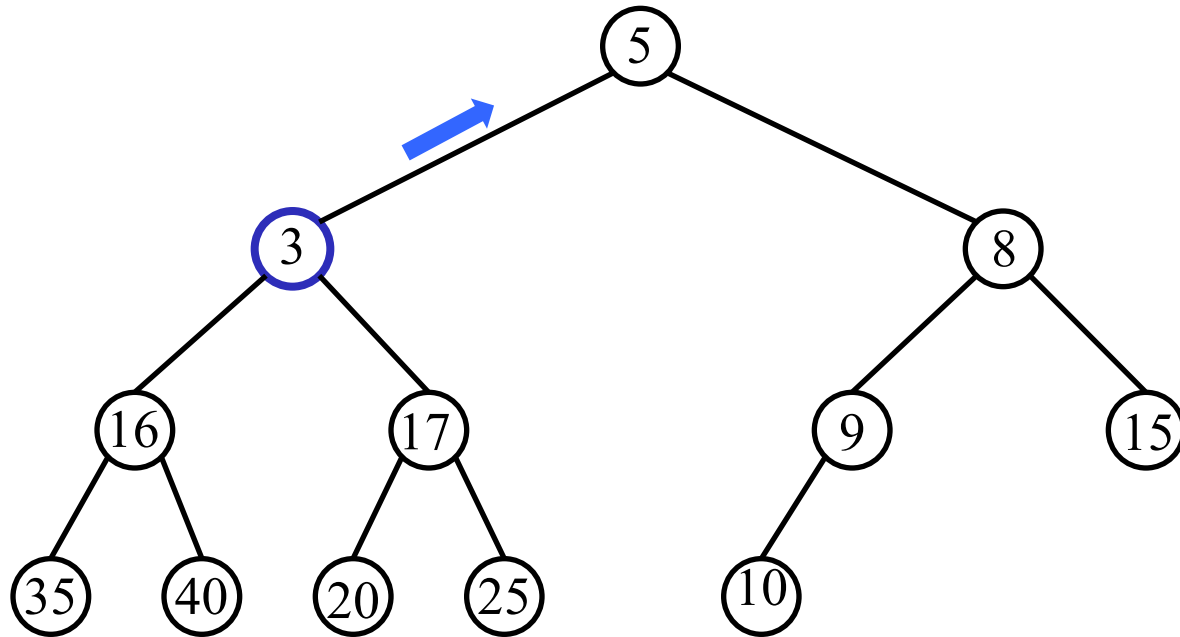
decreaseKey(elem e, chiave d)



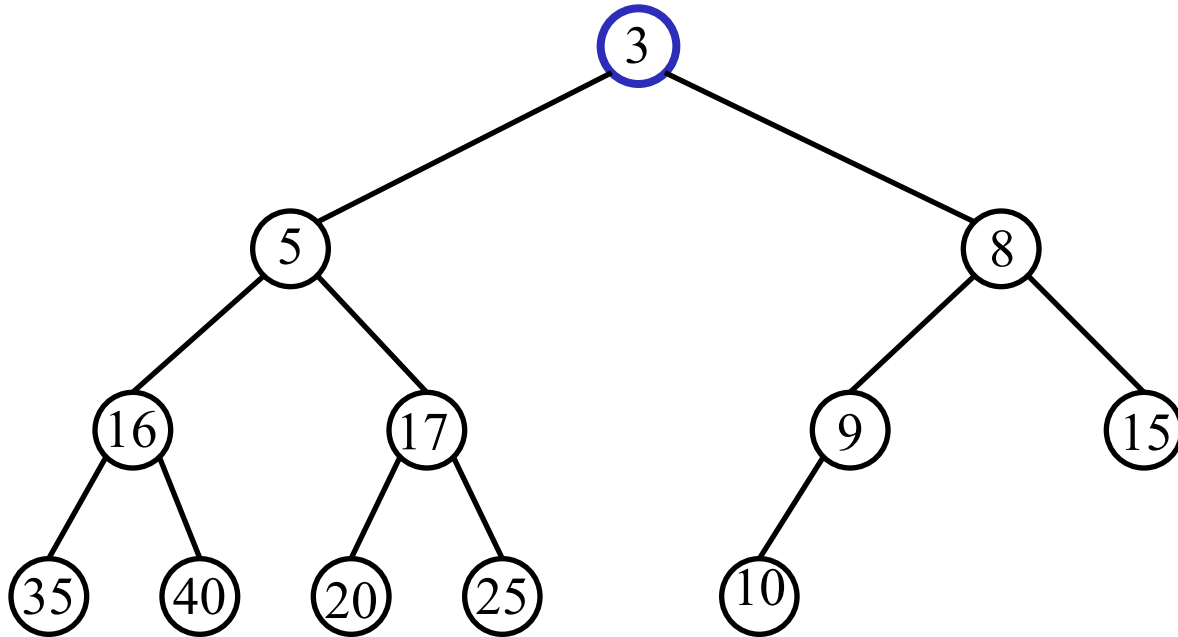
decreaseKey(elem e, chiave d)



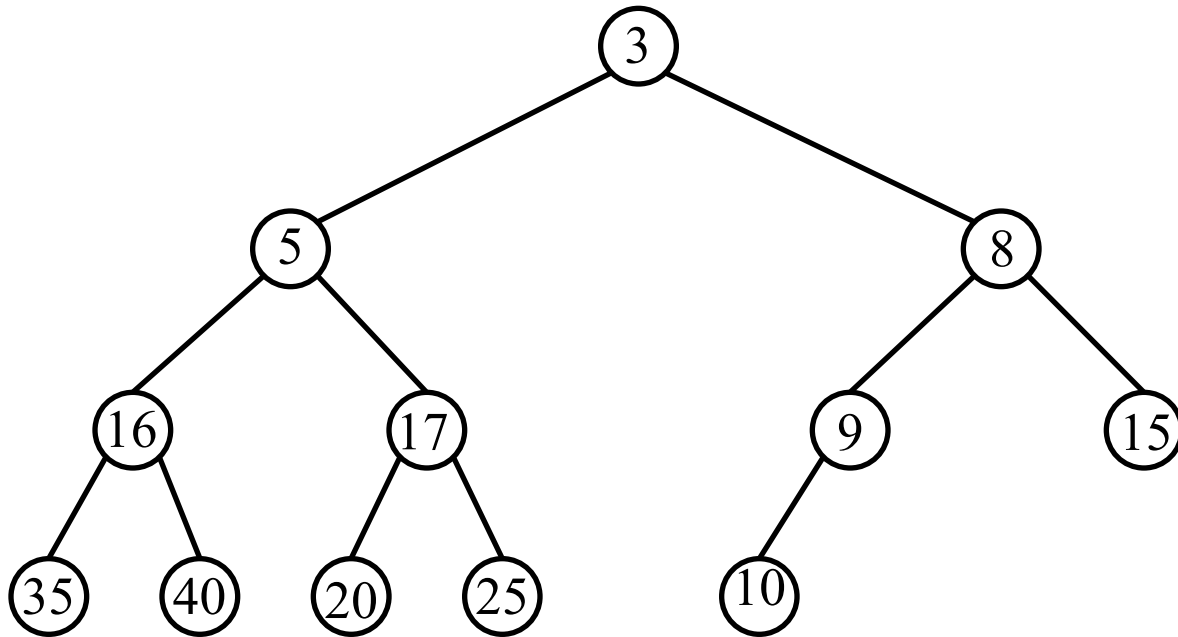
decreaseKey(elem e, chiave d)



decreaseKey(elem e, chiave d)



decreaseKey(elem e, chiave d)



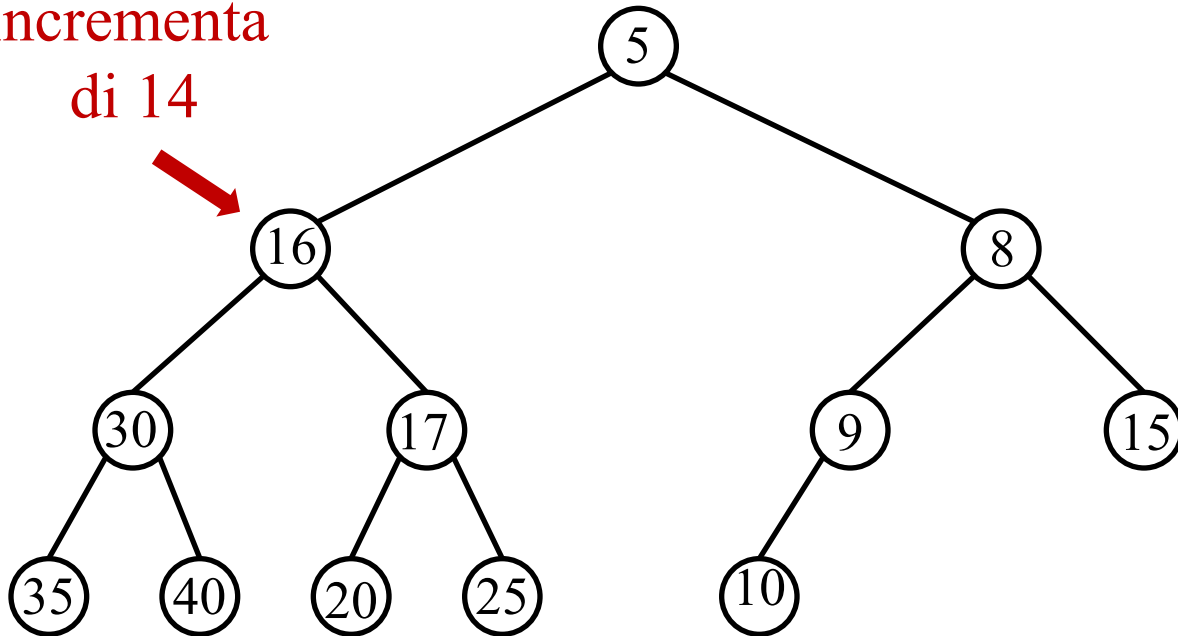
decreaseKey(elem e , chiave d)

decrementa il valore della chiave nel nodo v contenente l'elemento e della quantità richiesta d . Ripristina poi la proprietà dell'ordinamento a heap spingendo il nodo v verso l'alto tramite ripetuti scambi di nodi.

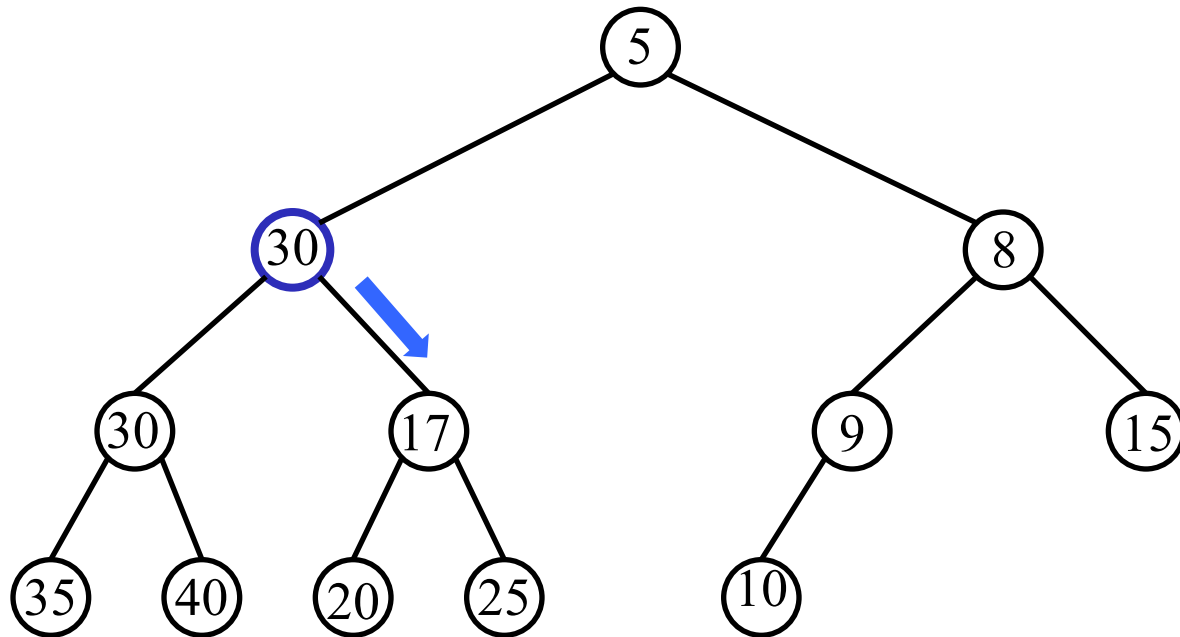
$T(n) = O(\log_d n)$ per l'esecuzione di muoviAlto

increaseKey(elem e, chiave d)

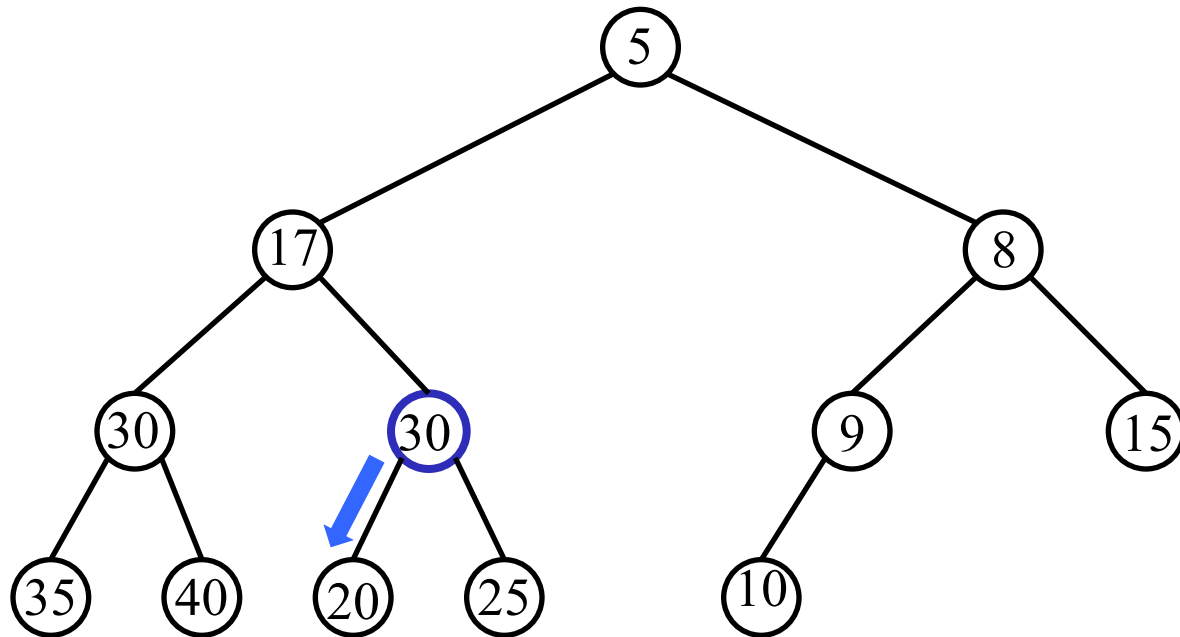
incrementa
di 14



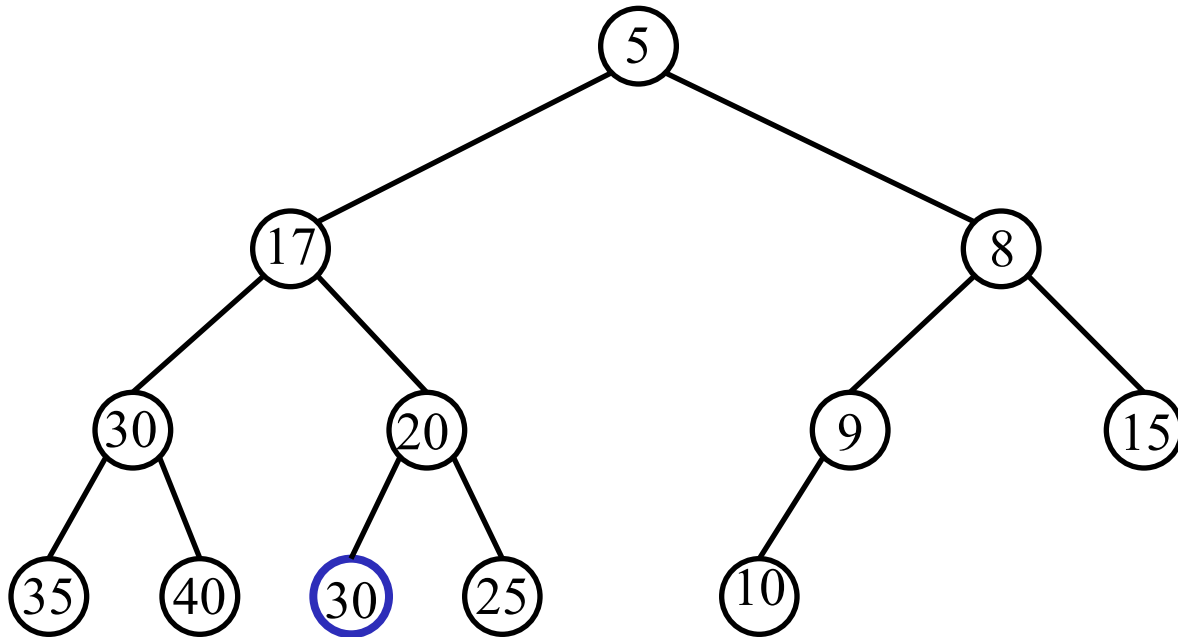
increaseKey(elem e, chiave d)



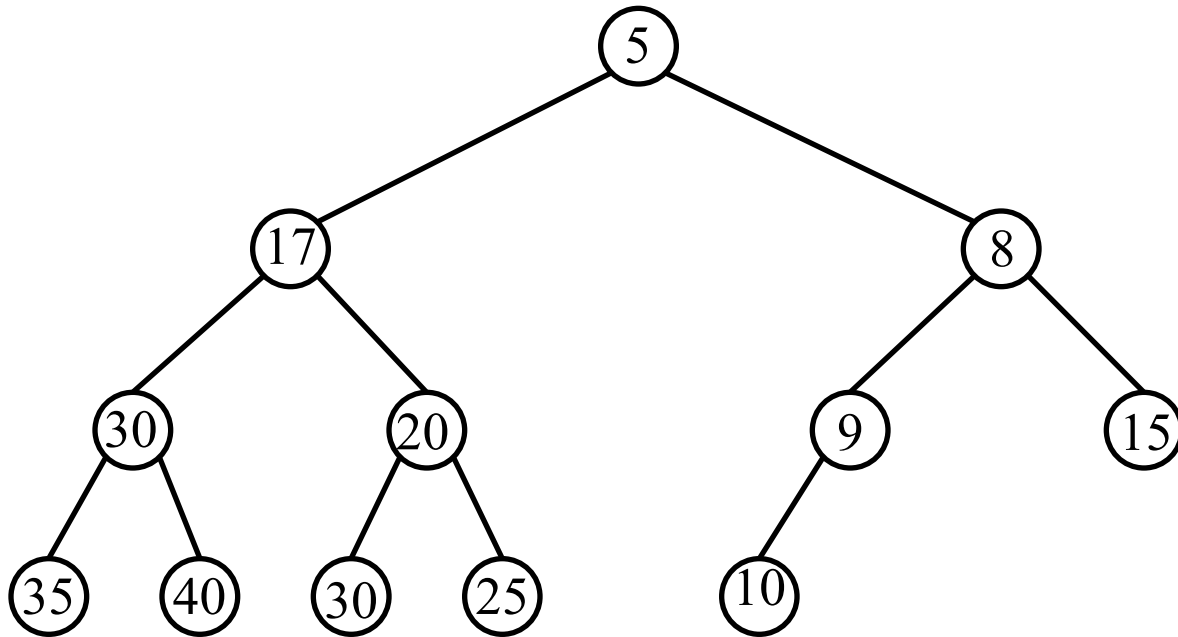
increaseKey(elem e, chiave d)



increaseKey(elem e, chiave d)



increaseKey(elem e, chiave d)



increaseKey(elem e , chiave d)

aumenta il valore della chiave nel nodo contenente l'elemento e della quantità richiesta d . Ripristina poi la proprietà dell'ordinamento a heap spingendo il nodo v verso il basso tramite ripetuti scambi di nodi.

$T(n)=O(d \log_d n)$ per l'esecuzione di muoviBasso

merge(CodaPriorità c_1 , CodaPriorità c_2)

Due modi:

1. **Costruire da zero**: si distruggono le due code e se ne crea una nuova contenente l'unione degli elementi.
2. **Inserimenti ripetuti**: si inseriscono ripetutamente gli elementi della coda più piccola in quella più grande.

Costruire da zero

Idea:

genero un nuovo heap d -ario contenente tutti gli elementi in c_1 e c_2

Come:

- generalizzazione della procedura **heapify**
- rendo i d sottoalberi della radice heap ricorsivamente e chiamo **muoviBasso** sulla radice

Complessità (d costante):

$$T(n) = d T(n/d) + O(d \log_d n) \quad \text{dove } n = |c_1| + |c_2|,$$



Teorema Master
(caso 1)

$$T(n) = \Theta(n)$$

Inserimenti ripetuti

Inseriamo ad uno ad uno tutti gli elementi della coda più piccola nella coda più grande.

Sia $k = \min\{|c_1|, |c_2|\}$ e $n = |c_1| + |c_2|$.

Eseguiamo quindi k inserimenti nella coda più grande.

Costo: $O(k \log n)$, dove $n = |c_1| + |c_2|$.

L'approccio conviene quindi per $k \log n = o(n)$, cioè per

$$k = o(n / \log n).$$

merge(CodaPriorità c_1 , CodaPriorità c_2)

Due modi:

1. **Costruire da zero**: si distruggono le due code e se ne crea una nuova contenente l'unione degli elementi.
2. **Inserimenti ripetuti**: si inseriscono ripetutamente gli elementi della coda più piccola in quella più grande.

Osservazione: nel caso peggiore entrambe le operazioni hanno un costo di $\Omega(n)$.

Algoritmi e Strutture Dati

Capitolo 8

Code con priorità:

Heap binomiali

Riepilogo

	Find Min	Insert	Delete	DelMin	Incr. Key	Decr. Key	merge
Array non ord.	$\Theta(n)$	$O(1)$	$O(1)$	$\Theta(n)$	$O(1)$	$O(1)$	$O(n)$
Array ordinato	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Lista non ordinata	$\Theta(n)$	$O(1)$	$O(1)$	$\Theta(n)$	$O(1)$	$O(1)$	$O(1)$
Lista ordinata	$O(1)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$
d-Heap	$O(1)$	$O(\log_d n)$	$O(d \log_d n)$	$O(d \log_d n)$	$O(d \log_d n)$	$O(\log_d n)$	$O(n)$

⇒ Il nostro obiettivo è implementare una coda di priorità con una struttura dati che non comporti **costi lineari**!

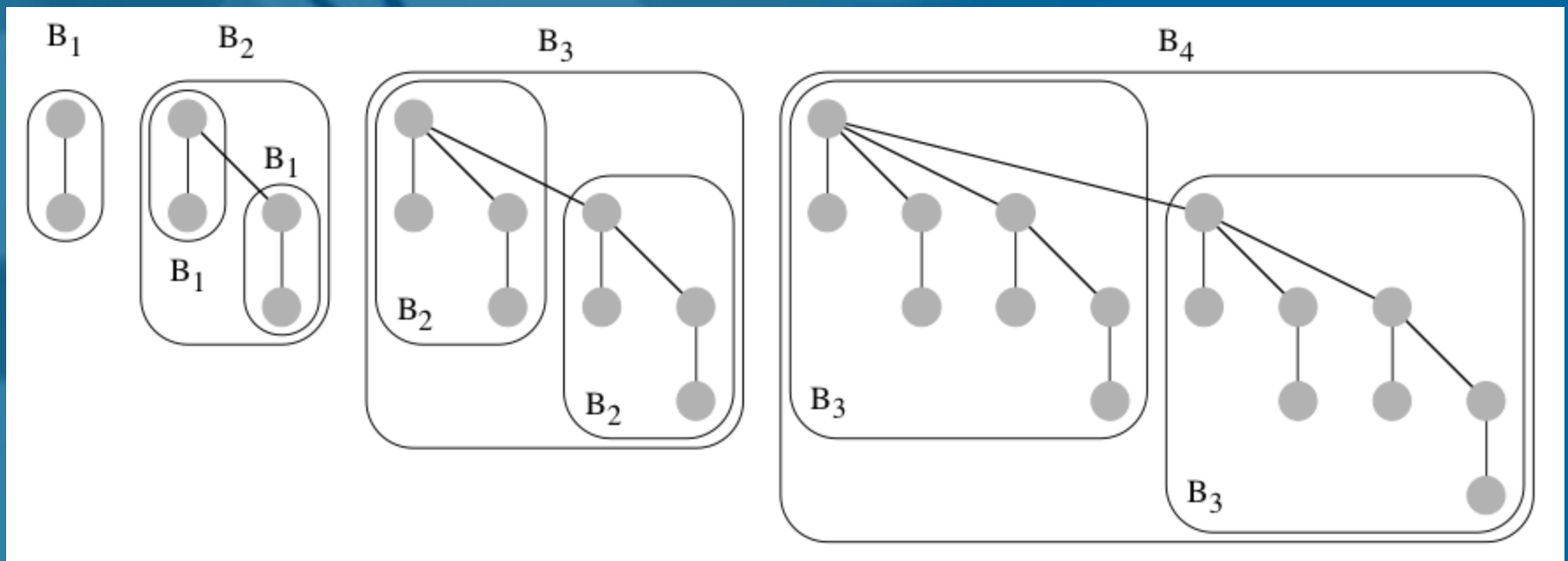


heap binomial

Alberi binomiali

Un **albero binomiale** B_i è definito ricorsivamente come segue:

1. B_0 consiste di un **unico** nodo
2. Per $i > 0$, B_{i+1} è ottenuto fondendo due alberi binomiali B_i , ponendo la radice dell'uno come figlia della radice dell'altro



Proprietà strutturali

Un albero binomiale B_h gode delle seguenti proprietà:

- 1. Numero di nodi ($|B_h|$): $n = 2^h$.*
- 2. Grado della radice: $D(n) = \log_2 n$*
- 3. Altezza: $H(n) = h = \log_2 n$.*
- 4. Figli della radice: i sottoalberi radicati nei figli della radice di B_h sono $B_0, B_1, \dots, B_{h-1}$.*

Dimostrazione per induzione su h

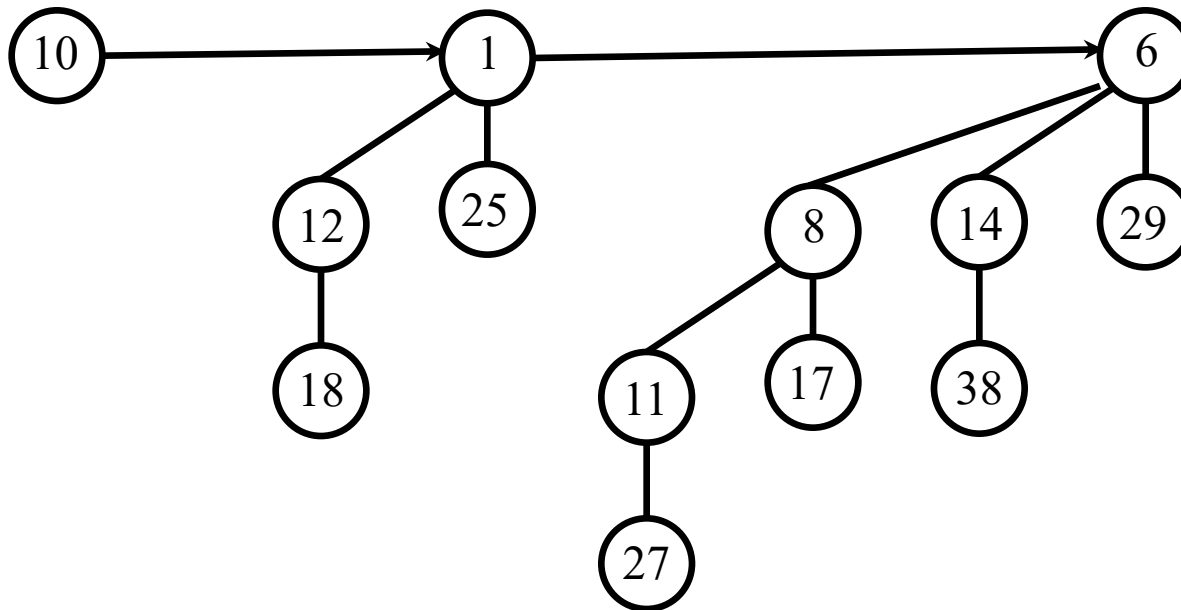
Definizione di heap binomiale

Un heap binomiale è una foresta (collezione) di alberi binomiali che gode delle seguenti proprietà:

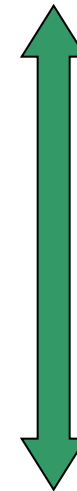
1. **Unicità:** per ogni intero $i \geq 0$, esiste al più un B_i nella foresta
2. **Contenuto informativo:** ogni nodo v contiene un elemento $\text{elem}(v)$ ed una chiave $\text{chiave}(v)$ presa da un dominio totalmente ordinato
3. **Ordinamento a heap:** $\text{chiave}(v) \geq \text{chiave}(\text{parent}(v))$ per ogni nodo v diverso da una delle radici

Un esempio di Heap
Binomiale con
 $n=13$ nodi

in questa direzione non è
presente un ordinamento

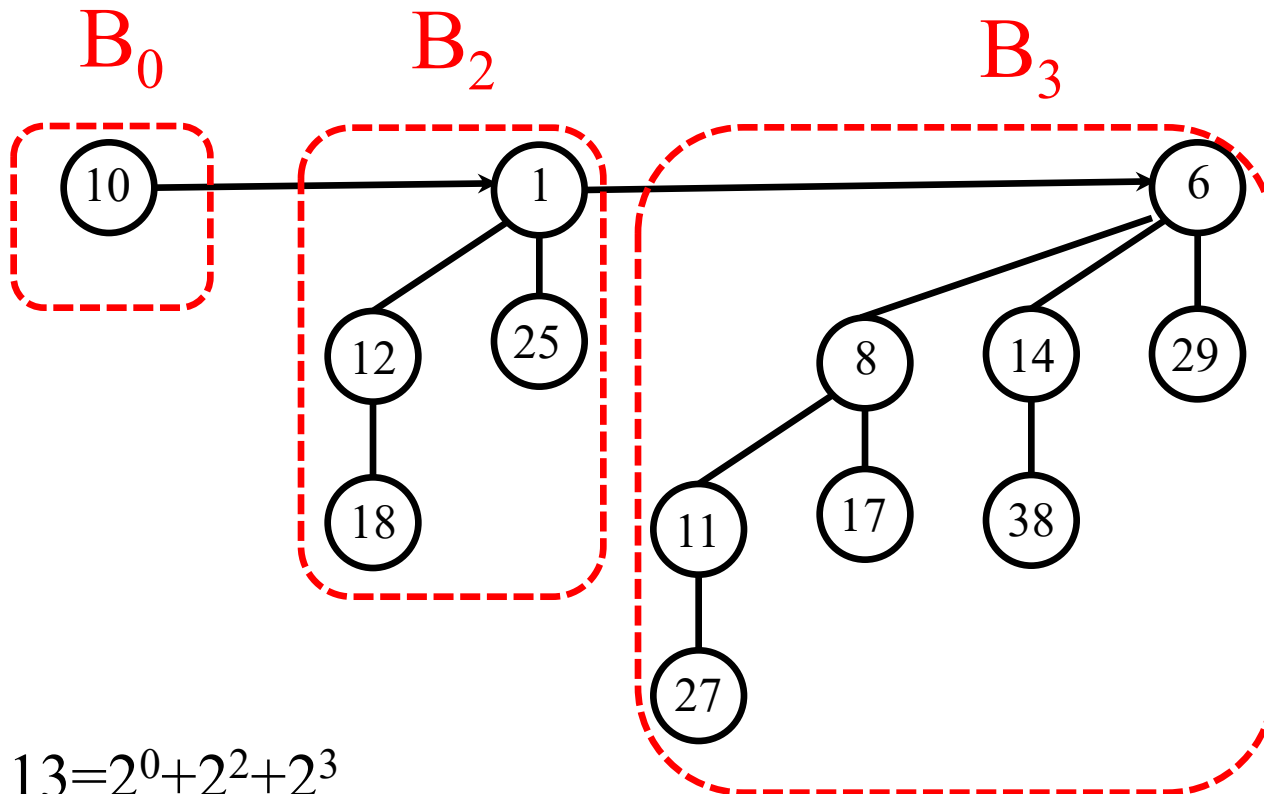


in questa
direzione è
presente un
ordinamento



domanda: quanti alberi binomiali può avere
al massimo un heap binomiale con n nodi?

Un esempio di Heap
Binomiale con
 $n=13$ nodi



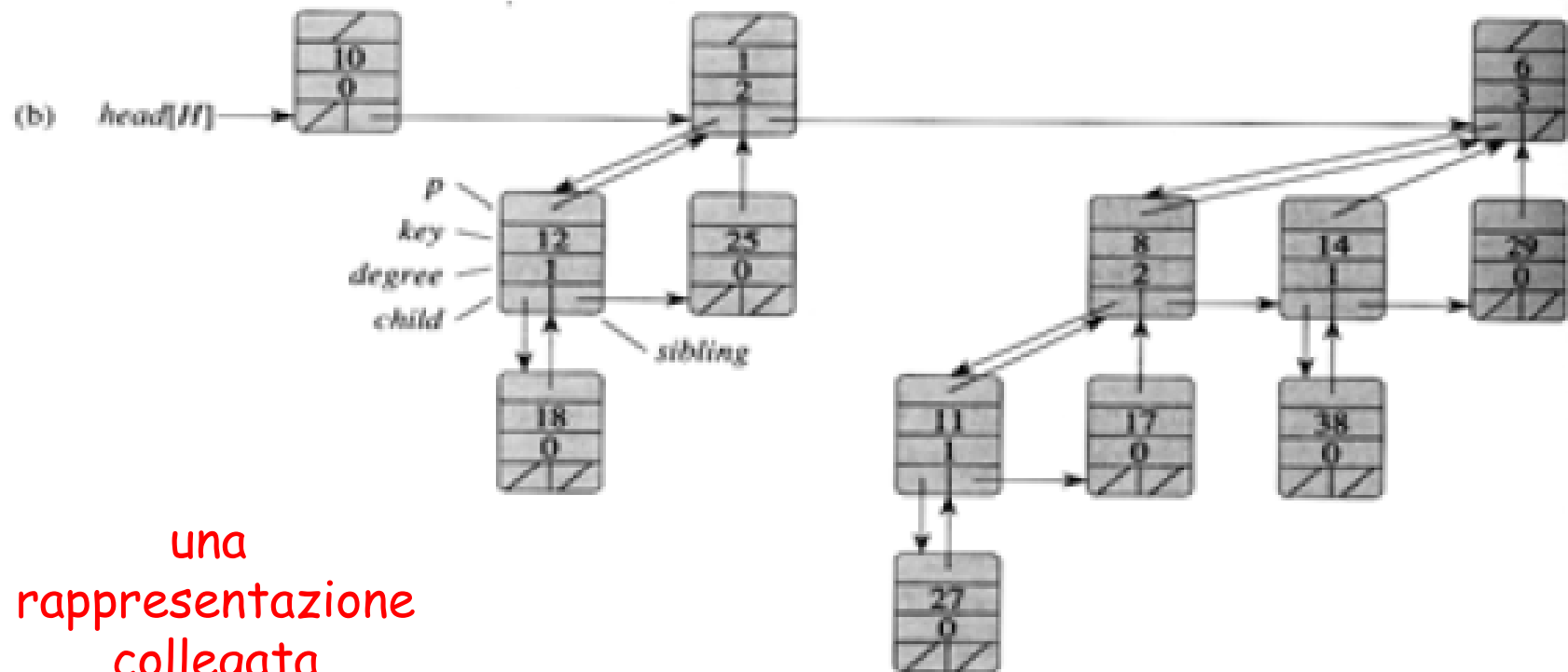
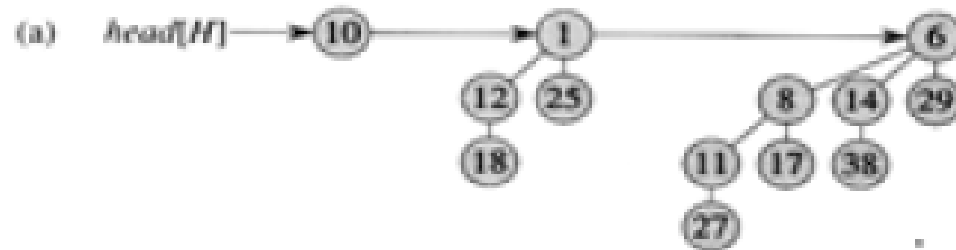
$$13 = 2^0 + 2^2 + 2^3$$

13 in binario: 1101

Proprietà topologiche

- Dalla proprietà di unicità degli alberi binomiali che lo costituiscono, ne deriva che un heap binomiale di n elementi è formato dagli alberi binomiali $B_{i_0}, B_{i_1}, \dots, B_{i_h}$, dove $i_0, i_1, \dots, i_h$ corrispondono alle posizioni degli 1 nella rappresentazione in base 2 di n .
- ⇒ Ne consegue che in un heap binomiale con n nodi, vi sono al più $\lceil \log n \rceil$ alberi binomiali, ciascuno con grado ed altezza $O(\log n)$

A Binomial Heap



una
rappresentazione
collegata

Procedura ausiliaria

Utile per ripristinare la proprietà di unicità in un heap binomiale (ipotizziamo di scorrere la lista delle radici da sinistra verso destra, in ordine crescente rispetto all'indice degli alberi binomiali)

```
procedura ristrutturare()
```

```
     $i = 0$ 
```

```
    while ( esistono ancora due  $B_i$  ) do
```

```
        si fondono i due  $B_i$  per formare un albero  $B_{i+1}$ , ponendo la radice con  
        chiave più piccola come genitore della radice con chiave più grande
```

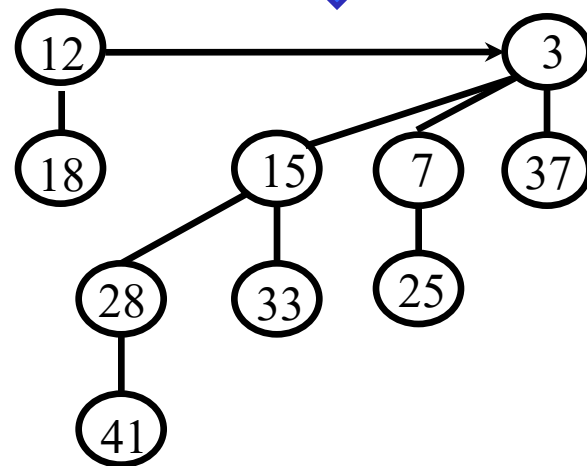
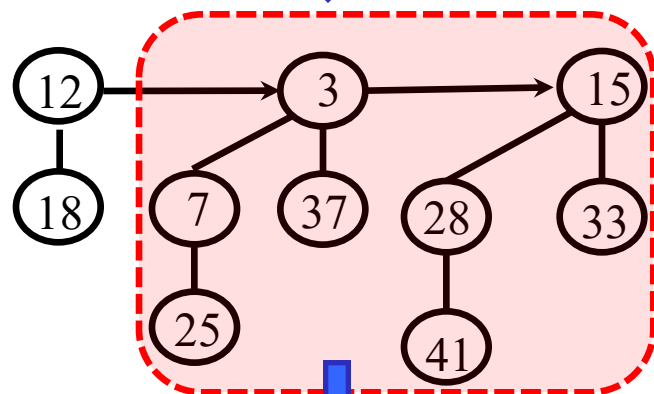
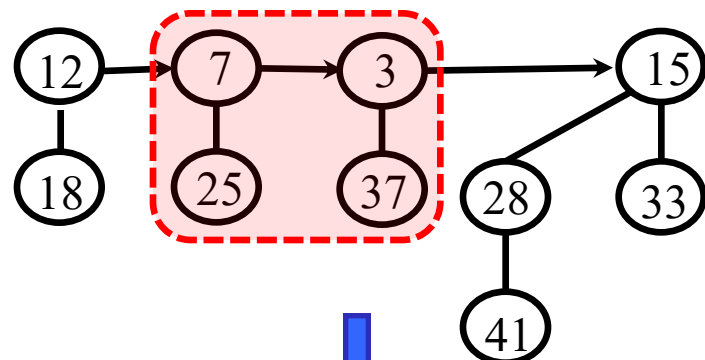
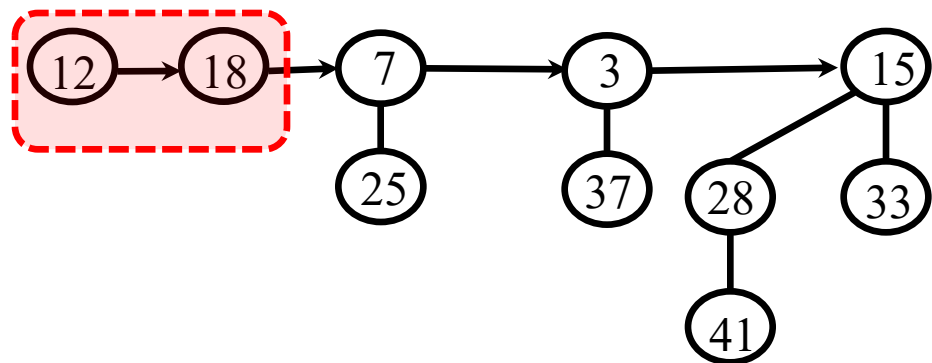
```
         $i = i + 1$ 
```

T(n): lineare nel numero di alberi binomiali in input

(ogni fusione diminuisce di uno il numero di alberi binomiali)

Ristruttura(H)

H



Realizzazione (1/3)

classe HeapBinomiale **implementa** CodaPriorita:
dati:

una foresta H con n nodi, ciascuno contenente un elemento di tipo $elem$ e una chiave di tipo $chiave$ presa da un universo totalmente ordinato.

operazioni:

`findMin()` $\rightarrow elem$

scorre le radici in H e restituisce l'elemento a chiave minima.

`insert( $elem\ e, chiave\ k$ )`

aggiunge ad H un nuovo B_0 con dati e e k . Ripristina poi la proprietà di unicità in H mediante fusioni successive dei doppioni B_i .

Realizzazione (2/3)

deleteMin()

trova l'albero T_h con radice a chiave minima. Togliendo la radice a T_h , esso si spezza in h alberi binomiali $T_0, \dots, T_{h-1}$, che vengono aggiunti ad H . Ripristina poi la proprietà di unicità in H mediante fusioni successive dei doppioni B_i .

decreaseKey(*elem* e , *chiave* d)

decrementa di d la chiave nel nodo v contenente l'elemento e . Ripristina poi la proprietà dell'ordinamento a heap spingendo il nodo v verso l'alto tramite ripetuti scambi di nodi.

delete(*elem* e)

richiama **decreaseKey**($e, -\infty$) e poi **deleteMin**().

Realizzazione (3/3)

`increaseKey(elem  $e$ , chiave  $d$ )`

richiama `delete( $e$ )` e poi `insert( $elem$ ,  $k + d$ )`, dove k è la chiave associata all'elemento e .

`merge(CodaPri.  $c_1$ , CodaPri.  $c_2$ )  $\rightarrow$  CodaPri.`

unisce gli alberi in c_1 e c_2 in un nuovo heap binomiale c_3 .

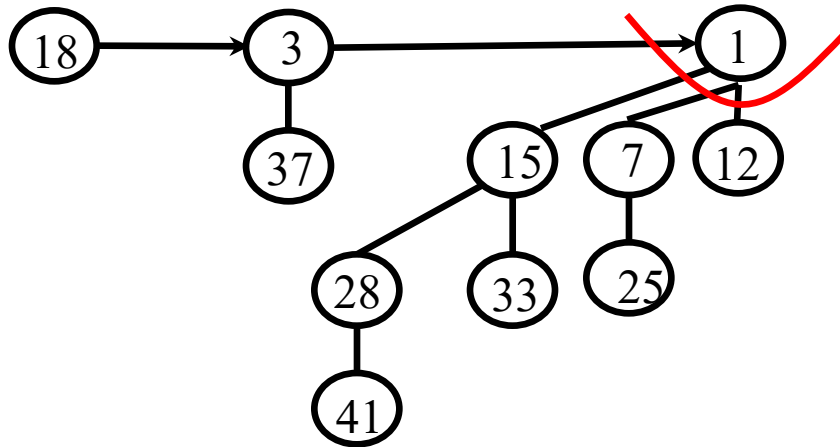
Ripristina poi la proprietà di unicità nell'heap binomiale c_3 mediante fusioni successive dei doppi B_i .

Tutte le operazioni richiedono tempo $T(n) = O(\log n)$

Durante l'esecuzione della procedura ristrutturazione esistono infatti al più tre B_i , per ogni $i \geq 0$

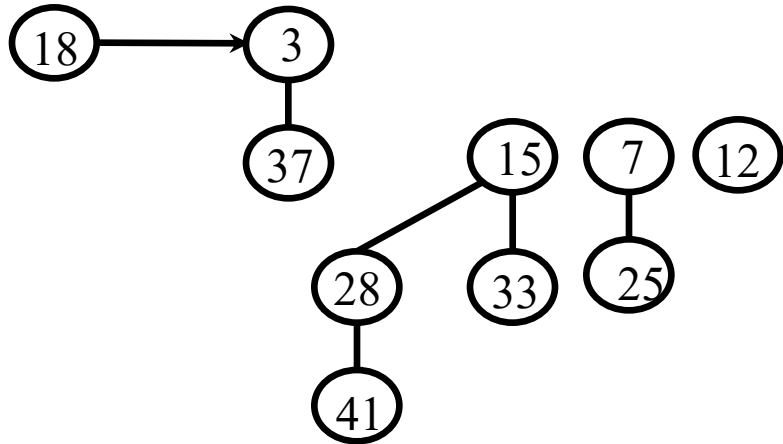
DeleteMin(H)

H



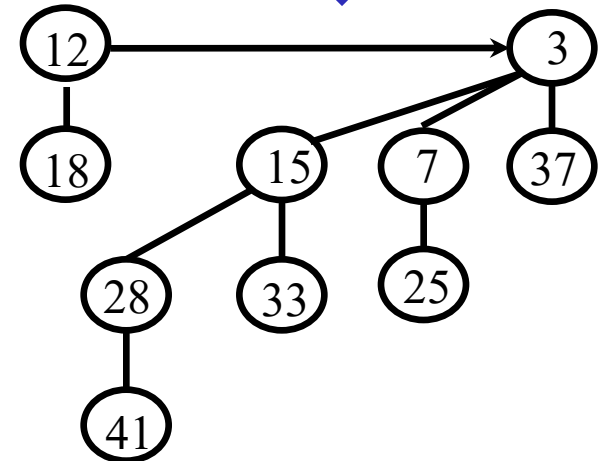
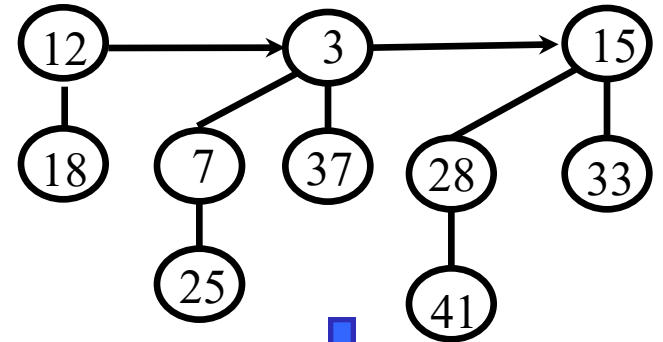
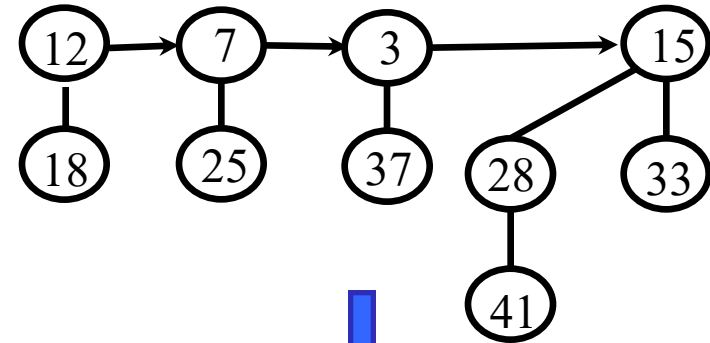
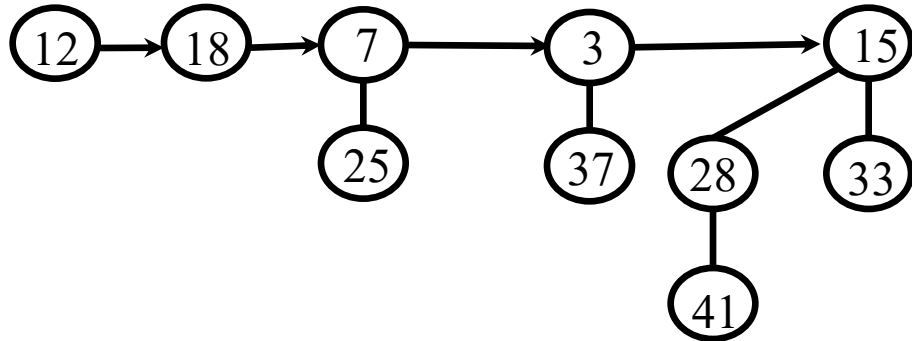
DeleteMin(H)

H

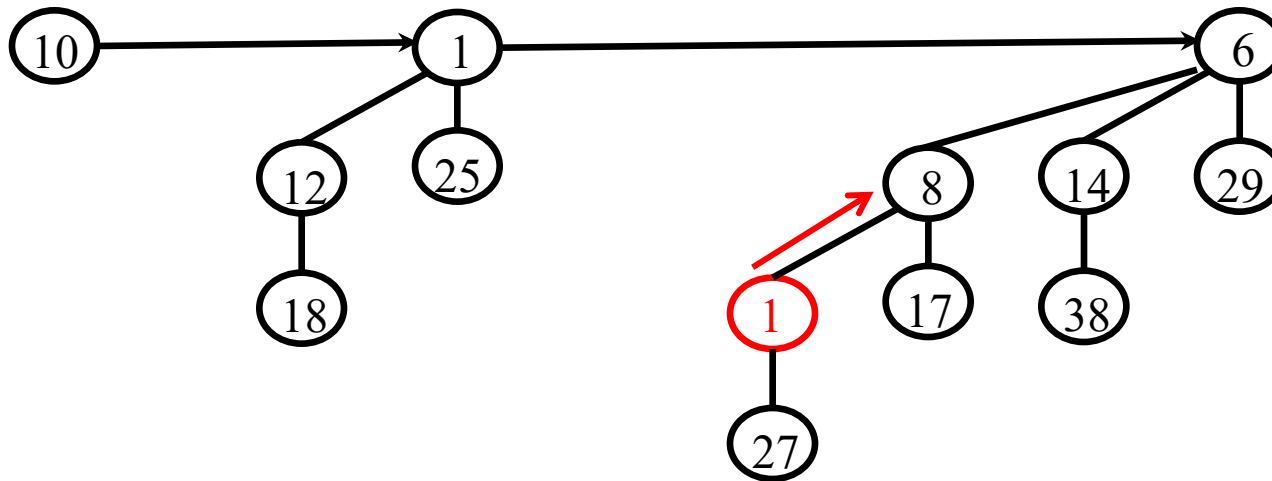
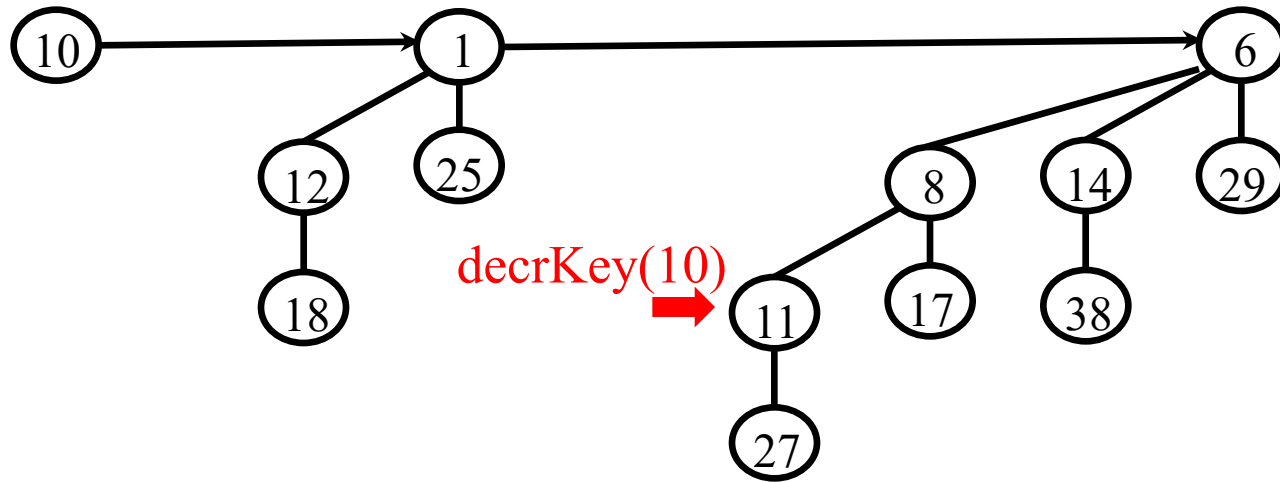


↓ fondi

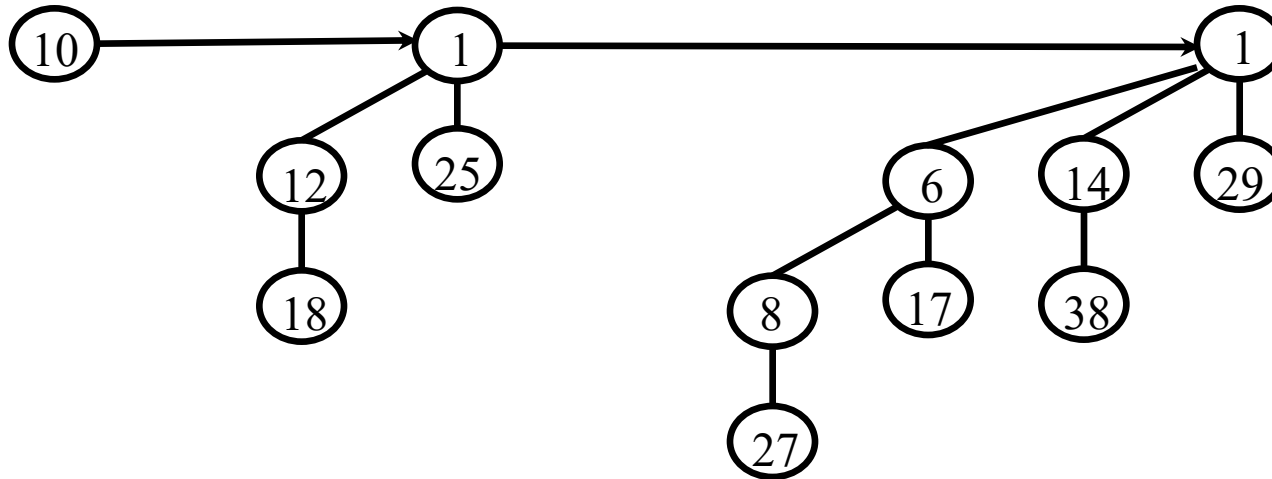
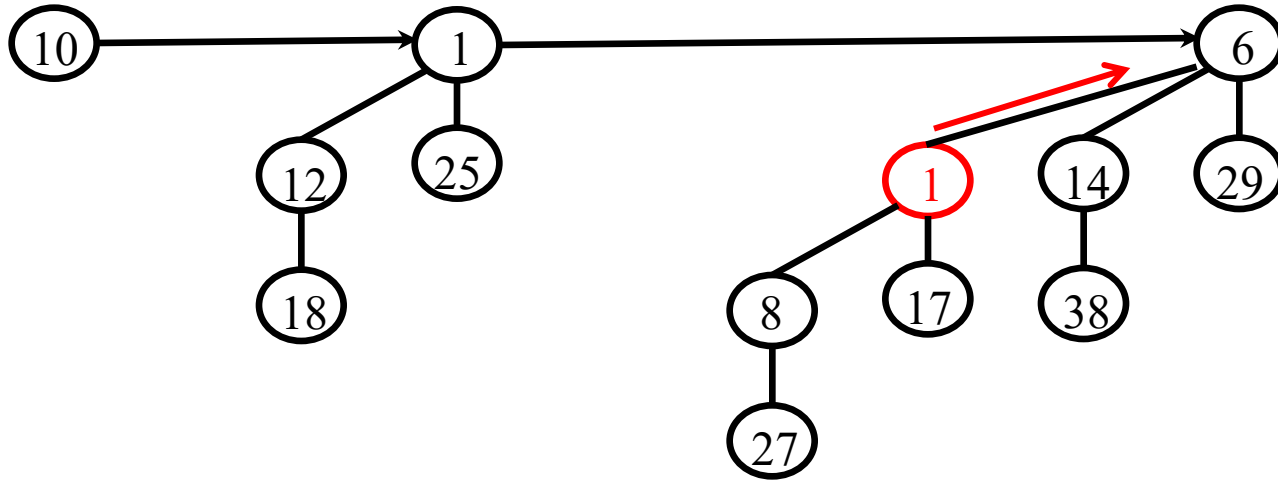
ristruttura

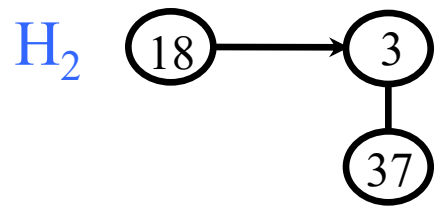
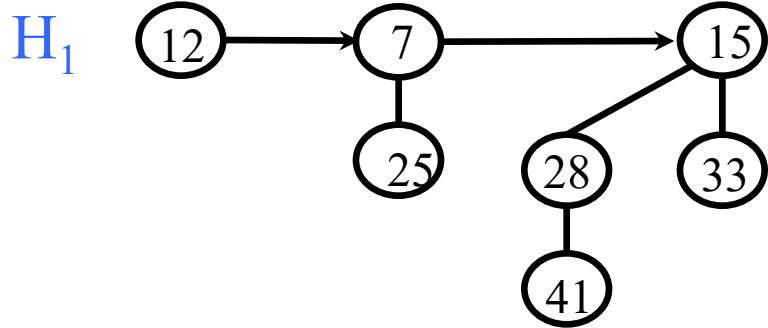


...decremento di una chiave



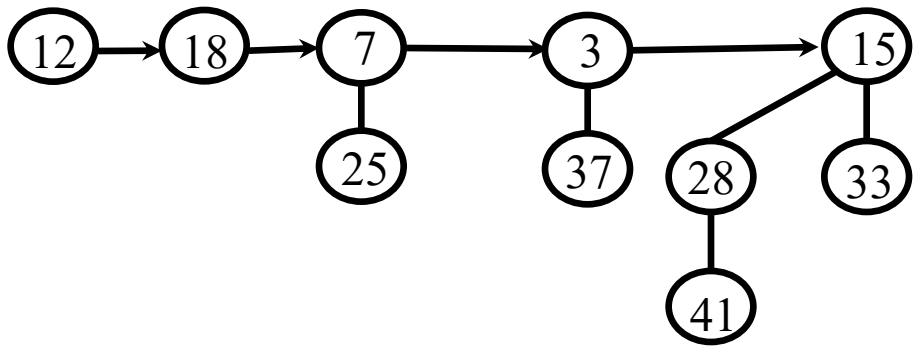
...decremento di una chiave



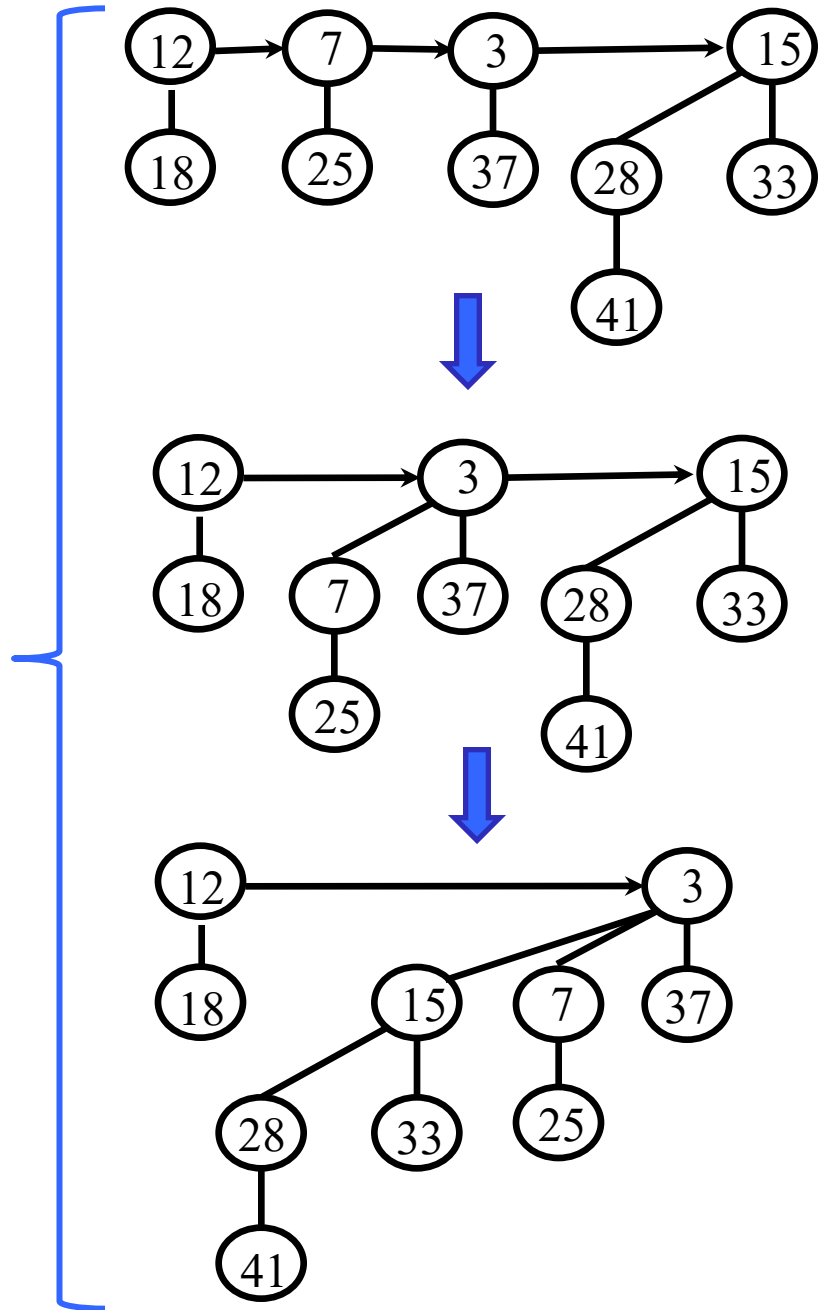


$\text{merge}(H_1, H_2)$

fondi



ristruttura





Heap di Fibonacci

(Fredman, Tarjan, 1987)

Heap di Fibonacci

Heap binomiale rilassato: si ottiene da un heap binomiale rilassando la proprietà di **unicità** dei B_i ed utilizzando un atteggiamento più “pigro” durante l’operazione insert (perché ristrutturare subito la foresta quando potremmo farlo dopo?)

Heap di Fibonacci: si ottiene da un **heap binomiale rilassato** indebolendo la proprietà di **struttura** dei B_i che non sono più necessariamente alberi binomiali

Analisi sofisticata: i tempi di esecuzione sono **ammortizzati** su sequenze di operazioni, cioè dividendo il **costo complessivo** della sequenza di operazioni per il numero di operazioni della sequenza

Conclusioni: tabella riassuntiva

	FindMin	Insert	Delete	DelMin	IncKey	DecKey	merge
d-Heap (d cost.)	$O(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Heap Binom.	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Heap Fibon.	$O(1)$	$O(1)$	$O(\log n)^*$	$O(\log n)^*$	$O(\log n)^*$	$O(1)^*$	$O(1)$

L'analisi per d-Heap e Heap Binomiali è nel caso peggiore, mentre quella per gli Heap di Fibonacci è ammortizzata (per le operazioni **asteriscate**)

Analisi ammortizzata

- Il **costo ammortizzato** di un'operazione è il costo “**medio**” rispetto a una **sequenza qualsiasi** di operazioni.
- **Esempio**: se un'operazione ha costo ammortizzato costante e eseguo una sequenza (qualsiasi) di **k** operazioni è possibile che il costo di una singola operazione può non essere costante, ma l'intera sequenza costerà **$O(k)$**
- Diverso dal **costo medio**: non c'è nessuna distribuzione di probabilità (sulla sequenza da eseguire) e l'algoritmo è un algoritmo deterministico
- Molto utile quando si vogliono **buone prestazioni sull'intera sequenza** e non garanzie sulla singola operazione
 - esempio: progettare algoritmi veloci attraverso strutture dati efficienti

...per esempio nel nostro caso:

Teorema

Usando un Heap di Fibonacci , una qualsiasi sequenza di n insert, d delete, f findMin, m deleteMin, Δ increaseKey, δ decreaseKey, μ merge prende tempo (nel caso peggiore)

$$O(n+f+\delta+\mu+(d+m+\Delta)\log n)$$

Esercizio (di manipolazione)

Creare ed unire 2 Heap Binomiali sui seguenti insiemi:

$$A_1 = \{3, 5, 7, 21, 2, 4\}$$

$$A_2 = \{1, 11, 6, 22, 13, 12, 23, 31\}$$